Release It!

Design and Deploy Production-Ready Software

Second Edition • Michael T. Nygard

Chapter-by-chapter reading notes

How to read these notes

- Headings are the book's own, word for word, in the book's order. A heading in the Contents but absent as a heading here is folded into bullets under its parent; sub-sections the book's Contents does not list are kept as lead-ins inside bullets.
- Content is only the author's argument, examples and numbers, in his words where he carries the point (exact short quotes) and close paraphrase otherwise. Nothing is added from outside the book.
- Blue box at the top of each chapter: Place in the argument — why this chapter, why this title. Gold box at the end: How this chapter builds the author's case. Each Part opens with an overview page.
- These notes are a first reading. Density is capped at 1 character of notes for every 3 of book, per chapter, so detail is cut first; "see 'Section'" points to what to read in the book.
- Not summarised: Acknowledgments, Bibliography, Index, and the code listings inside chapters.

Contents

Preface 4
1. Living in Production 5
Part I — Create Stability (overview) 7
2. Case Study: The Exception That Grounded an Airline 8
3. Stabilize Your System 10
4. Stability Antipatterns 12
5. Stability Patterns 24
Part II — Design for Production (overview) 32
6. Case Study: Phenomenal Cosmic Powers, Itty-Bitty Living Space 33
7. Foundations 36
8. Processes on Machines 39
9. Interconnect 42
10. Control Plane 47
11. Security 52
Part III — Deliver Your System (overview) 57
12. Case Study: Waiting for Godot 58
13. Design for Deployment 59
14. Handling Versions 63
Part IV — Solve Systemic Problems (overview) 66
15. Case Study: Trampled by Your Own Customers 67
16. Adaptation 70
17. Chaos Engineering 77

The Architecture of the Book at a Glance

Part	Author's stated aim	Chapters
Chapter 1 (before Part I)	Sets the target: "life in production", not passing QA	1 Living in Production
Part I — Create Stability	Shows how to "keep your systems alive, maintaining system uptime."	2 to 5
Part II — Design for Production	Ongoing operations in complex virtualized, containerized, load-balanced environments; patterns for control, transparency, availability	6 to 11
Part III — Deliver Your System	Deployment: frequent small changes without breaking consumers; no-downtime deployment; versioning across services	12 to 14
Part IV — Solve Systemic Problems	The system's ongoing life: evolutionary architecture, shared knowledge across systems, antifragile systems via chaos engineering	15 to 17

Overall purpose, in the author's words (Preface): "In this book, you will examine ways to architect, design, and build software—particularly distributed systems—for the muck and mire of the real world." "Your software will be under attack from the moment you release it." The book looks "at software that failed the test" to "find ways to make sure your software survives contact with the real world."

Preface

Place in the argument — why this chapter, why this title. Promise and reader first, then the road-map: stability comes first because “There’s no viable future without stability”.

Chapter opening — untitled section before the first sub-heading in the book

- Book on building distributed systems “for the muck and mire of the real world”; it studies software that failed so yours survives.

Who Should Read This Book

- Architects, designers, developers of systems that “must be available or the company loses money”.

How This Book Is Organized

- Four parts, each introduced by a case study.
- Part I, Create Stability: keep systems alive. Distributed systems show availability like “two eights” rather than “five nines”; short-term fixes dominate an unstable system.
- Part II, Design for Production: ongoing operations; complex virtualized, containerized, load-balanced environments; patterns for control, transparency, availability.
- Part III, Deliver Your System: deployment. Pouring bits onto servers is easy; frequent small changes without breaking consumers is hard. Design for deployment, no-downtime deployment, versioning across services.
- Part IV, Solve Systemic Problems: the system’s ongoing life (1.0 is birth): evolutionary architecture, shared knowledge across systems, antifragile systems via chaos engineering.

About the Case Studies

- Real failures he observed; names obfuscated, but sequence, failure mode, propagation, outcome and costs kept. “Real money is on the line when systems fail.”

Online Resources

- Book web page: source code, forums, errata.

How this chapter builds the author’s case. Sets the order (stability, operations, delivery, evolution) the book follows.

CHAPTER 1

Living in Production

> **Place in the argument — why this chapter, why this title.** The title names the target: life in production, not passing QA. It comes first to set the aim, then the argument runs target, scale, money, early decisions, kind of architect, then hands to the airline case study.

Chapter opening — untitled section before the first sub-heading in the book

- "Feature complete" is not "production ready". Design as taught covers what systems should do, not what they should not do: "crash, hang, lose data, violate privacy, lose money, destroy your company, or kill your customers."
- Teams aim to pass QA instead of life in production, but real-world stresses go beyond what you could ever test for.
- Accept that bad things will still happen: prevent what you can, and ensure the system "as a whole can recover" from unanticipated traumas.

Aiming for the Right Target

- Most software is designed for the lab and QA tests, "the artificial realm of QA, not the real world of production".
- Analogy: early-90s car design in the lab was elegant but fragile and short-lived. You want a car designed by someone who knows oil changes are always late and you will stomp the brakes holding an Egg McMuffin and a phone.
- Passing QA says little about the next three to ten years: Toyota Camry, Chevy Vega or Ford Pinto of software?
- "Design for manufacturability" fixed designs thrown over the wall to production. Our analog is "design for production": we hand finished software to IT operations, and must design systems and the ecosystem to operate at low cost and high quality.

The Scope of the Challenge

- Client/server had tens or hundreds of users; now user counts exceed continents, and even commerce sites are expected 24 by 7 by 365 (once "five nines" was mainframe territory).
- More reach and scale bring "new ways to break, more hostile environments, and less tolerance for defects". Small-website designs fail outrageously on large transactional distributed systems.

A Million Dollars Here, a Million Dollars There

- Systems built for QA cost so much in operations, downtime and maintenance that they never reach profitability.
- Optimizing development cost at operational cost suits a team with a fixed budget, not the organization. Systems spend far more life in operation.
- Example: 5 minutes downtime per release, monthly, five years = 300 minutes at $3,000/minute, about $1,000,000. A $50,000 build pipeline avoiding it is a 2,000 percent ROI (before faster deployment and market share).
- "Design and architecture decisions are also financial decisions": the fusion of technical and financial views is a recurring theme.

Use the Force

- Early decisions shape the system most and are hardest to reverse (team structure, funding, time-sheet codes): "Team assignments are the first draft of the architecture." Yet they are the least informed.

- He backs agile: production is the only place to learn, so begin learning early. Still, decisions need foresight (“use the force”), since alternatives with similar implementation cost have radically different life-cycle costs for availability, capacity and flexibility. Examples come from real systems.

Pragmatic Architecture

- Two kinds of architecture. The ivory tower pursues abstraction and decrees (“The middleware shall be JBoss, now and forever!”); an architect who ignores coders ignores users too, and users cheer when the system crashes.
- The pragmatic architect is a coder, peels back abstractions, discusses memory, CPU, bandwidth.
- Ivory tower wants an end-state of perfection; pragmatic thinks about the “dynamics of change” (deploy without rebooting the world, which metrics, what to improve), with components good enough for current stresses.
- Pragmatic architects get ammunition; ivory-tower ones may descend to “living in production”.

Wrapping Up

- “Software delivers its value in production”; everything before is prelude. The book covers life in production; Part I is stability, starting with the bug that grounded an airline.

> **How this chapter builds the author’s case.** Fixes the target (production) and the financial stakes, leads into Part I via the airline case study (chapter 2).

PART I OVERVIEW

Part I — Create Stability

Author's description (Preface): "Part I: Create Stability shows you how to keep your systems alive, maintaining system uptime." Distributed systems show availability like "two eights" rather than "five nines". "Stability is a necessary prerequisite to any other concerns." "There's no viable future without stability, so we'll start by looking at ways to make a stable base."

The theme of the Part

- Keep the system alive: stability comes first because an unstable system makes short-term fixes and short-term thinking dominate everything else.

How the chapters build the theme

- Chapter 2, Case Study: the airline grounded by one exception shows the stakes in a real story.
- Chapter 3, Stabilize Your System: turns the story into vocabulary and principles (stability, failure modes, the chain of failure).
- Chapter 4, Stability Antipatterns: the forces that create, accelerate or multiply cracks.
- Chapter 5, Stability Patterns: the remedies that answer the antipatterns.

What the Part contributes to the book

- A base of stable instances; Part II builds the environment they live in.

CHAPTER 2

Case Study: The Exception That Grounded an Airline

Place in the argument — why this chapter, why this title. Per the preface, each part is introduced by a case study; this opens Part I (stability) as ch01 promised (the software bug that grounded an airline). The title states the lesson: something very small, one exception, caused a very big outage. Sub-headings follow the order of events, then investigation: change, outage, consequences, postmortem, clues, smoking gun, prevention.

Chapter opening — untitled section before the first sub-heading in the book

- Incidents that blow up start with something very small; "A tiny programming error starts the snowball rolling downhill." Here it stranded thousands of passengers and cost hundreds of thousands of dollars. Names, places and dates are changed.
- It began with a planned failover on the database cluster serving the core facilities (CF), a first-generation service in the airline's move to service-oriented architecture, rolled out in phases.
- CF handled flight searches. Check-in kiosks, phone menus (IVR) and channel-partner feeds used CF; gate agent and call center applications did not yet (a good thing).
- Built for high availability: J2EE application servers, redundant Oracle 9i database, RAID array with off-site backups and replicas, Veritas Cluster Server managing the active database and virtual IP, redundant hardware load balancers, redundant hardware throughout, racks split, a second site thirty miles away. "CF did not suffer from any of the usual single-point-of-failure problems."

The Change Window

- A local engineering team had run the infrastructure for three years. On a Thursday around 11 p.m. Pacific they did a manual failover from database 1 to database 2 via Veritas for routine maintenance, then back, done dozens of times before.
- Veritas could do it in about a minute, and the application servers (connected to the virtual IP only) could not tell. Monitoring showed continuous availability. About 12:30 a.m. the change was marked "Completed, Success"; the engineer, after a 22-hour shift, went to bed.
- "Nothing unusual occurred until two hours later."

The Outage

- About 2:30 a.m. every kiosk went red nationwide, then the IVR servers: 5:30 a.m. Eastern, prime commuter check-in time. A Severity 1 case was opened.
- His principle: restoring service takes precedence over investigation. Pre-written scripts for thread dumps and database snapshots are the right balance: "It's not improvised, it does not prolong an outage, yet it aids postmortem analysis."
- Restoring service means figuring out what to target; rebooting the world works but is slow. Like diagnosing a fever, symptoms alone are not specific enough.
- Two apps hung at nearly the same time, so a common dependency was suspected: CF, the only shared one, which also had a failover three hours earlier. Monitoring showed nothing because it only hit a status page ("did not really say much about the real health").
- Near the one-hour SLA, they restarted the CF application servers: IVR recovered; restarting the kiosks' own servers fixed kiosks. Total: a little over three hours.

Consequences

- The impact lasted far longer. Without kiosks, off-shift agents were called in (overtime), clearing the backlog only by nearly 3 p.m. Early flights could not push back, gates were occupied, incoming flights reassigned gates. It reached Good Morning America and the Weather Channel.
- The FAA report card and complaints affect the CEO's compensation.

Postmortem

- Eight hours in, he was called to fly in to find why the database failover caused the outage. “Post hoc, ergo propter hoc” (“you touched it last”) is a good starting point, not always right.
- Agenda: did the failover cause it, was the cluster configured correctly, was maintenance done correctly, how could it have been detected, and how to make sure it never happens again. His presence also managed perception.
- “A postmortem is like a murder mystery”: reliable clues (logs) and unreliable ones (people mix observations with speculation), and the body is gone because servers are back up.
- He planned to collect logs, thread dumps and configs, and compare current configs with the nightly backup to see whether anyone covered up a mistake.

Hunting for Clues

- The cluster and RAID configuration were fine: “Proven, textbook work.”
- Application server logs held thread dumps (“the application is an open book, if you know how to read it”: libraries, thread pools, protocols). (see the boxed ‘Getting Thread Dumps’ for how.)
- Kiosk server: all forty request threads were blocked in SocketInputStream.socketRead0(), reading a response that never came, under FlightSearch.lookupByCity(), via RMI and EJB.
- RMI “can be dangerous because calls cannot be made to time out”, leaving the caller vulnerable to the remote server.

The Smoking Gun

- CF used separate thread pools for EJB and HTTP, so it answered monitoring while every EJB thread, on every server, was blocked checking out a database connection. This was circumstantial evidence only.
- No source access, so he decompiled production binaries. The lookupByCity() method closes its Statement and Connection in a finally block (see ‘The Smoking Gun’ for the code). It looks well constructed but has a fatal flaw: Statement.close() can throw SQLException, and then the connection is not closed.
- After failover, pooled JDBC connections were dead (TCP state did not carry over), yet the driver still created statements; executing and closing threw. Each call leaked a connection; after forty, the pool was exhausted and all calls blocked at getConnection().
- The JDBC specification allows close() to throw, “so your code has to handle it.”
- “The entire globe-spanning, multibillion dollar airline … was grounded by one programmer’s error: a single uncaught SQLException.”

An Ounce of Prevention?

- Code review would catch it only with knowledge of Oracle’s driver internals; more testing perhaps, as a stress test reproduced it, but the regular test profile did not exercise the method enough.
- “Bugs will happen. They cannot be eliminated, so they must be survived instead.”
- The worst problem: the bug propagated to other systems. The better question: “How do we prevent bugs in one system from affecting everything else?” Interconnected systems “must not” allow chains of failure; the next chapters look at design patterns for this.

> **How this chapter builds the author’s case.** Shows the stakes from ch01 concretely: a tiny bug, no timeouts and a chain of failure across systems. It sets up ch03 (stability, chain of failure) and the antipatterns and patterns of Part I.

CHAPTER 3

Stabilize Your System

> **Place in the argument — why this chapter, why this title.** Right after the airline case study, this chapter turns the story into vocabulary and principles. The title is an imperative: stability is the first job. Sub-headings run from definitions, to the danger of longevity, to failure modes, to the airline's missed stopping points, to the chain of failure, then on to Chapters 4 and 5.

Chapter opening — untitled section before the first sub-heading in the book

- New software is like a new graduate facing the real world; lab tests are "contrived by people who know what answer they expect to get".
- "Enterprise software must be cynical": it expects bad things, does not trust itself, and "refuses to get too intimate with other systems".
- The airline's Core Facilities was not cynical enough; dazzled by new technology, it "didn't see the stop sign".
- Poor stability has costs: lost revenue (the Chapter 1 retailer loses $1,000,000 per hour); wasted marketing (up to $150 to acquire a customer; 5,000 visitors per hour with 10 percent leaving for good is $75,000); and lost reputation.
- "Good stability does not necessarily cost a lot": at leverage points, the highly stable design usually costs the same to implement as the unstable one.

Defining Stability

- A transaction is an abstract unit of work (not a database transaction), e.g. "customer places order". A system is the complete interdependent set of hardware, applications and services needed to process transactions.
- A robust system keeps processing transactions despite transient impulses, persistent stresses, or component failures: the user can still get work done.
- Impulse is a rapid shock, like a hammer: a flash mob, a celebrity tweet. Stress is a force applied over an extended period, e.g. slow responses from a credit card processor.
- Stress produces strain, which can propagate and appear as distant effects (RAM on web servers, I/O on the database).
- Longevity: keeping processing for a long time; a working definition of a long time is the time between code deployments.

Extending Your Life Span

- Major dangers to longevity: memory leaks and data growth; rarely caught in testing. By Murphy's Law, whatever you do not test against will happen.
- Development and QA servers get recycled, are not under continuous load; load tests run for a set period and quit.
- Remedy: run your own longevity tests, e.g. a spare machine running a load tool continuously, with slack hours to catch connection pool and firewall timeouts. If the economics do not justify it, test the important parts and stub the rest.
- Otherwise production becomes the longevity test.

Failure Modes

- Impulses and strain both trigger catastrophic failure: some component starts failing first. Chiles (Inviting Disaster) calls these "cracks in the system", like a crack in a steel plate propagating faster under stress. The trigger, spread and damage together are a **failure mode**.

- Systems will have failure modes; denying it “robs you of your power to control and contain them”. Like crumple zones, design safe failure modes that contain damage; this self-protection determines resilience.
- Chiles calls these protections “crackstoppers”: decide which features are indispensable and keep cracks away. If you do not design failure modes, you get unpredictable and dangerous ones.

Stopping Crack Propagation

- Applied to the airline: the crack started at the improper SQLException handling but could have been stopped at many points, from low-level to architecture.
- Pool: it blocked requesting threads forever; it could create more connections or block callers for a limited time.
- Calls: RMI calls never time out by default; clients could set socket timeouts (Socket.setSoTimeout()), CF could have used HTTP web services with request timeouts, or callers could jettison blocked threads. None was done.
- Partition CF into service groups so one group’s problem does not take down all users.
- Architecture: request/reply message queues (caller knows a reply might never arrive). “The more tightly coupled the architecture, the greater the chance this coding error can propagate”; less-coupled architectures are shock absorbers.
- The designers had not considered “cracks” when creating shared services.

Chain of Failure

- Underneath every outage is a chain of events: one small issue leads to another. The chain looks improbable only if events are treated as independent, like coin tosses; they are not, because layers are coupled (a slow database makes application servers more likely to run out of memory).
- Terms: **fault** (condition creating an incorrect internal state), **error** (visibly incorrect behavior), **failure** (an unresponsive system, “in the eye of the beholder”). “Triggering a fault opens the crack. Faults become errors, and errors provoke failures.”
- At each step the crack may accelerate, slow or stop; complexity and coupling add pathways. “Tight coupling accelerates cracks”: EJB calls let CF’s resource exhaustion hurt callers, and request-handling threads coupled to external calls turned a remote problem into downtime.
- Preparing for everything means asking of every external call, I/O and resource “What are all the ways this can go wrong?” (no connection, ten minutes to connect, disconnect, no response, 10,000 simultaneous requests). Brute force is impractical except for life-critical systems.
- Two camps: make systems fault-tolerant (catch exceptions, keep faults from becoming errors), or say that is futile (“let it crash” and restart from a known good state). Both agree faults will happen and must be kept from becoming errors; you must decide whether to risk failure or errors. Patterns will provide shock absorbers.

Wrapping Up

- Every production failure is unique, but patterns emerge: the **stability antipatterns** (Chapter 4). Common solutions exist: design and architecture patterns (Chapter 5).
- These patterns “cannot prevent cracks in the system. Nothing can.” They stop cracks propagating, contain damage and preserve partial functionality instead of total failure.
- First the bad news: the antipatterns that will kill your systems.

> **How this chapter builds the author’s case.** Reuses Chapter 2’s airline and Chapter 1’s costs to define stability, introduces cracks, failure modes and the fault, error, failure chain, and sets up Chapter 4 (antipatterns) then Chapter 5 (patterns).

CHAPTER 4

Stability Antipatterns

Place in the argument — why this chapter, why this title. The title names "antipatterns that can wreck your system": "common forces that have contributed to more than one system failure". Each one will "create, accelerate, or multiply cracks in the system". The chapter sits before the patterns on purpose: the next chapter covers patterns that "alleviate or prevent the antipatterns from harming your system", but "we need to understand what we're up against" first. The sub-headings are the antipatterns themselves, and the author himself ranks several of them: Integration Points are "the number-one source of cracks", Cascading Failures "the number-one crack accelerator", Blocked Threads "the proximate cause of most failures". Slow Responses and Unbounded Result Sets sit at the end and point back to earlier ones (blocking, handshaking, steady state). Every section closes with a "Remember This" list of lessons and names the stability pattern that counters the antipattern.

Chapter opening — untitled section before the first sub-heading in the book

- The "software crisis" (coined at the first NATO Software Engineering Conference, 1968) never ended, even though machines, languages, libraries, open source and the number of programmers have improved enormously, because "we've steadily taken on bigger and bigger challenges".
- Scale has shifted: once a hundred active users was large, then ten thousand (first edition), now millions and even a billion-user site (Facebook, 1.13 billion daily active users in 2016). An application is now dozens or hundreds of services, run continuously and redeployed continuously.
- So even five nines "is nowhere near enough"; at that scale it still means thousands of disappointed users a day. Six Sigma quality on Facebook "would create 768,000 angry users per day" (200 requests per page, 1.13 billion users, 3.4 defects per million).
- Reach has also exploded: everything within the enterprise is interconnected, then integrated across enterprises, and application boundaries are fuzzy as features go to SaaS services.
- Bigger challenges mean "tightly coupled systems are the rule rather than the exception". Big systems serve more users by commanding more resources, but "in many failure modes big systems fail faster than small systems".
- Following James R. Chiles (Inviting Disaster), these systems sit at the "technology frontier", where "high interactive complexity and tight coupling conspire to turn rapidly moving cracks into full-blown failures".
- High interactive complexity: so many moving parts and hidden dependencies that operators' mental models are incomplete or wrong, so their instinctive actions range "from ineffective to actively harmful". Hidden linkages cause "problem inflation" (a minor fault becomes a major failure); Chiles blames them partly for Three Mile Island. They look obvious in the postmortem but are "devilishly difficult to anticipate".
- Tight coupling lets cracks propagate or multiply across layer or system boundaries. A failure in one component redistributes load to its peers and delays and stresses its callers, making another failure more likely, "eventually resulting in total collapse". It can occur in application code, in calls between systems, or "any place a resource has multiple consumers".
- The chapter looks at antipatterns, bad behaviours to avoid. Avoiding them is not enough: "Everything breaks. Faults are unavoidable." Don't pretend you can eliminate every source of faults; "Assume the worst", and examine what happens after the fault creeps in (the patterns of the next chapter).

Integration Points

- Author's starting claim: he hasn't seen a "straight-up website" project since about 1996; everything is an integration project (HTML veneer, front-end app, API, mobile app). Context diagrams are one of two shapes: the butterfly (a central system with feeds fanning in on one side and a large fan out on the other; not necessarily a bad monolith, maybe a well-factored system with a lot of responsibility)

or the spiderweb (many boxes and dependencies, orderly if you were diligent and lucky, chaotic like a black widow's if not).

- What they share: connections outnumber services. A butterfly has 2N connections; a spiderweb might have up to N squared. All of these connections are integration points, and "every single one of them is out to destroy your system". The trend toward many smaller services, SaaS providers and API-first makes it worse.
- Case, You Have How Many Feeds?: launching a retailer's replatform, the author asked for the production firewall rules for feeds in and out. Nobody could say what all the feeds were; a project manager dedicated to integration needed a database to track them. The site was plagued by stability problems at launch (3 a.m. crashes); "every single synchronous integration point caused at least one outage".
- Conclusion drawn: "Integration points are the number-one killer of systems." Every socket, process, pipe or remote procedure call "can and will hang", even database calls. Every feed can hang or crash the system or send impulses at the worst moment.
- Socket-Based Protocols: nearly everything above named pipes and shared memory runs on sockets, so all higher protocols inherit socket-layer failures. The simplest failure, connection refused, is well handled because every language API makes it explicit. The wrinkle is that discovering you can't connect can take a long time.
- Why: the arrow in an architecture diagram abstracts a network connection, itself an abstraction over packets. TCP's three-way handshake (SYN, SYN/ACK, ACK) builds the connection. If nobody listens, a reset returns in under ten milliseconds. If the remote application is hammered, the port's listen queue fills; the calling thread is then blocked inside the OS kernel in open(), and connection timeouts "usually measured in minutes".
- Similarly, read() can block until the server responds, often forever by default; you must set the socket timeout and handle the exception.
- Network failures come "fast or slow". Fast ones (connection refused) throw immediately. Slow ones (a dropped ACK) block threads for minutes; if all block, the server is down. Hence: "Clearly, a slow response is a lot worse than no response."
- Case, The 5 A.M. Problem: a site on about thirty instances hung completely at almost exactly 5 a.m. daily, just as traffic ramped up; restarts cured it. Thread dumps showed every request-handling thread blocked in low-level socket reads/writes inside the Oracle JDBC library. Packet capture showed a few packets from app servers to the database with no replies, and no traffic at all on the database side of the firewall though the database was healthy: "the absence of activity was the biggest clue".
- Cause, in order: a TCP connection exists only as state in the endpoints' memory and can persist for days without packets. A firewall (a specialised router with ACL rules) keeps a finite table of established connections plus a last packet time and drops idle ones (here after one hour), but TCP has no way for a third party to tell the endpoints. This firewall also silently dropped later packets (no ICMP reply, as a security setting). The stacks then retransmitted for ages: tcp_retries2 default 15 gave a twenty-minute timeout on Linux, thirty minutes on the author's HP-UX servers, and reads could block forever.
- The resource pool used last-in, first-out: overnight one connection was reused and the other thirty-nine sat idle beyond the firewall's one-hour limit. When traffic ramped up, those dead connections locked up threads at once, and sooner or later the one good connection was held by a blocked thread: total site hang.
- Fix considered: pool validity tests (SELECT SYSDATE FROM DUAL) or tearing down old connections would themselves hang; a reaper thread was too complex. A DBA suggested Oracle's dead connection detection, whose periodic ping from server to client resets the firewall's last packet time and keeps the connection alive.

- Lesson: “not every problem can be solved at the level of abstraction where it manifests.” You must be able to drill through at least two layers of abstraction. (See the sidebar ‘Packet Capture’ for tcpdump and Wireshark.)
- HTTP Protocols: REST with JSON over HTTP is the lingua franca; it has all socket problems plus its own, centred on client libraries. The provider may accept the connection but never respond; accept but not read the request (a large body fills the TCP window and even the socket write blocks); send a status the caller can’t handle; send an unexpected content type (e.g. an HTML 404 page, or an ISP-injected page); or claim JSON while sending something else.
- Advice: use a client library with fine-grained control of connection timeout, read timeout and response handling, and avoid libraries that map responses directly into domain objects. Treat a response as data until confirmed (revisited in Chapter 11, Security).
- Vendor API Libraries: vendors’ server software may be hardened, but their client libraries are “just code coming from regular developers” with the usual variability, and you have little control over them (decompile, report bugs, press the vendor). “The prime stability killer … is all about blocking” (resource pools, socket reads, HTTP connections, serialization).
- Example (see ‘Vendor API Libraries’ for the Java listings): a callback interface (UserCallback.messageReceived) plus Connection.registerCallback/send. You can’t tell from the code which thread calls the callback or what locks it already holds, so deadlock is a minefield; an implementation may synchronize the callback, blocking library threads and then senders, tying up request-handling threads.
- Countering Integration Point Problems: the most effective patterns are Circuit Breaker and Decoupling Middleware. Testing helps too: “Cynical software should handle violations of form and function”. Build a test harness (simulator with controllable behaviour) for each integration, with canned responses for functional testing and switches for network and system failures, then flip them under considerable load, which takes more than a handful of testers.
- Remember This: every integration point will eventually fail, and not through nice error responses but through protocol violations, slow responses or hangs; know when to open up abstractions (packet sniffers); failures propagate quickly and cascade if your code isn’t defensive; apply Circuit Breaker, Timeouts, Decoupling Middleware and Handshaking.

Chain Reactions

- Setting: the dominant style is the horizontally scaled farm of commodity hardware (vertical scaling, bigger servers, has its place but most interactive load goes to farms). Identical servers behind a load balancer give fault tolerance through redundancy, so a single machine or process can bonk while the rest keep serving.
- Yet such clusters, safe from single points of failure (except self-denial attacks), have a load-related failure mode: when one node fails the others pick up the slack. Eight servers each carry 12.5 percent; after one fails, seven carry about 14.3 percent. That is only 1.8 percent more of the total workload, but each server’s own load rises about 15 percent. In a two-node cluster the survivor’s workload doubles.
- If the first failure came from a load-related condition (memory leak, intermittent race condition) the survivors become more likely to fail, and with each loss the rest are “more and more likely to also go dark”.
- Definition: “A chain reaction occurs when an application has some defect—usually a resource leak or a load-related crash.” The layer is homogeneous, so every server carries the defect; the only cure is to fix it. Splitting a layer into pools (Bulkhead) can only turn one chain reaction into two at different rates.
- Effects: a chain reaction in one layer easily causes a cascading failure in a calling layer. Chain reactions are sometimes caused by blocked threads: when all request-handling threads of an application block, requests are redistributed to the others in the layer, raising their chance of failure.

- Case, Searching…: a retailer with half a million SKUs relied on a dozen search engines behind a hardware load balancer with health checks. A vendor memory leak made all engines go dark around noon on normal traffic days; each lost engine pushed its share to the others, so they ran out of memory faster. The “last response” timestamps showed an accelerating pattern (gaps of five or six minutes, then three or four, then seconds). Losing the last engine locked up the whole front end (cascading failure and blocked threads). Until the vendor patch arrived months later, daily restarts at 11 a.m., 4 p.m. and 9 p.m. bracketed peak hours.
- Remember This: one server down jeopardizes the rest; hunt for resource leaks (most often memory) and obscure timing bugs; use autoscaling with health checks for every group, as long as the scaler reacts faster than the chain reaction propagates; defend with Bulkheads, which don’t help callers of the partition that goes down, so use Circuit Breaker on the calling side.

Cascading Failures

- Framing: failures start with a crack from some fundamental problem (a latent bug triggered by the environment, a memory leak, an overloaded component). Slowing the crack is the topic of the next chapter; without that, structural problems amplify it. Definition: “A cascading failure occurs when a crack in one layer triggers a crack in a calling layer.”
- Example: an entire database cluster goes dark; what happens next depends on how the caller is written, and if it handles it badly the caller fails too. Problems cascade upward (as trees drawn upside-down).
- Modern systems are dozens or hundreds of interlinked services, each with its own database, “Every dependency is a chance for a failure to cascade”. Crucial services with high fan-in (many callers) spread their problems widely and deserve extra scrutiny.
- Cascades need a mechanism for the failure to “jump the gap” when the caller misbehaves in response to the provider’s failure. Common: resource pools drained by a failure below; “Integration points without timeouts are a surefire way to create cascading failures.”
- Layer-jumping usually takes the form of blocked threads, but the author has also seen an overly aggressive caller: it got quick errors and, by historical precedent, retried, assuming they were transient errors from a race condition. The lower layer didn’t give enough detail to tell transient from serious errors, so when a failed switch really dropped database packets the caller pounded harder, until it used 100 percent of its CPU calling and logging failures. A Circuit Breaker would have helped. Speculative retries also let failures jump the gap, since a slow provider triggers more retries that tie up more caller threads.
- Ranking claim: “Just as integration points are the number-one source of cracks, cascading failures are the number-one crack accelerator.” Preventing them “is the very key to resilience”; the most effective patterns are Circuit Breaker and Timeouts.
- Remember This: stop cracks jumping the gap (they can follow a chain reaction below; make sure you stay up when other enterprise systems go down); scrutinise resource pools, which must always limit how long a thread waits to check out a resource; defend with Timeouts (so you can come back from a call) and Circuit Breaker (avoid calling the troubled point).

Users

- Opening: “Users are a terrible thing. Systems would be much better off with no users.” He says it tongue-in-cheek: users are why systems exist, but they have “a knack for creative destruction” (the seagull landing on the hood of the car teetering on the cliff edge), and calling systems “march remorselessly forward like an army of Terminators”.
- Traffic: eventually traffic exceeds capacity (if it isn’t growing you have other problems); the question is how the system reacts to excess demand. Capacity is the maximum throughput sustainable under a given workload while maintaining acceptable performance. Hard internal limits create cracks, and “cracks always propagate faster under stress”. In the cloud, autoscaling is a friend, but buggy applications can run up a huge bill.

- Heap Memory: web back ends give every user a session that stays resident for a time after the last request (the dead time before session timeout), and every object in it ties up bytes. Short memory brings surprising failures: at best an out-of-memory error to the user, at worst even logging fails (an argument for external monitoring in addition to log scraping), and a recoverable low-memory situation turns into a serious stability problem.
- Advice: keep as little in the session as possible (don't keep a whole search result set for pagination; requery per page) because data may never be used again. Weak or soft references (SoftReference in Java, weakref in Python, WeakReference in C#) let the garbage collector reclaim the payload under memory pressure, usually before an out-of-memory error. They are not transparent: accessors must handle a null payload by recomputing, redirecting or protecting. They add complexity, so "it's best to just keep things out of the session". (See 'Heap Memory' for the code fragment.)
- Off-Heap Memory, Off-Host Memory: move per-user memory to another process (memcached, Redis). It trades addressable size against latency; remote memory can beat local disk but local memory beats remote, so "There's no one-size-fits-all answer."
- Sockets: every active request uses an open socket on an ephemeral port; 16-bit port numbers give at most 16,383 connections in the IANA range, or 64,511 using ports 1024-65535. Servers with a million connections rely on virtual IP addresses (each with its own port range), which needs application changes, kernel buffers and TCP tuning.
- Closed Sockets: closed sockets wait in TIME_WAIT before reuse, as defence against bogons (late stray packets). Inside your data center that is less likely an issue, so you can turn TIME_WAIT down.
- Expensive to Serve: most retail users browse a couple of pages (cached, no integration points), but the buyer needs four or five checkout pages and several integration points (card authorisation, tax, address standardisation, inventory, shipping). More buyers stress the front end and downstream systems (see Unbalanced Capacities). "There is no effective defense against expensive users", and he doesn't recommend keeping them off, since they generate revenue. Instead test aggressively: double or triple the proportion of expensive transactions (a 2 percent conversion rate tested at 4, 6 or 10 percent). 100 percent conversion is a fine stability test but not for capacity planning, or you "will spend ten times too much on hardware".
- Unwanted Users, innocent first: a badly configured proxy server re-requested one legitimate user's last URL, accelerating from every thirty seconds to four or five per second, carrying the user cookie but no session cookie, so each request created a new session. Sessions are "the Achilles' heel of web applications": requesting a deep link without cookies and dropping the socket makes the app server build the response anyway and create a session (kilobytes), so even a desktop can generate hundreds of thousands of sessions. Worse, the last login interceptor treated each request as a login: profile loads and updates of one row, so 100,000 transactions updating the same row deadlocked and one hung transaction (waiting on a connection from a different pool) blocked the rest, until every request thread was used up and the site was down.
- Unwanted Users, crafty: competitive intelligence companies leech data page by page, like a battalion of clipboard-carrying shoppers crowding out real ones, and they ignore session cookies, so new sessions pile up. robots.txt is "a social convention—not even a standard", so the robots that respect it are those that bring revenue, and the leeches ignore it. Only two approaches work: technical (identify and block scrapers at the CDN or outer firewalls, using ARIN and noticing inconsistent user-agents, and expire old blocks periodically; "This is a form of Circuit Breaker") and legal (terms of use limiting content to personal or noncommercial use, then lawyers). Neither is permanent: "pest control".
- Malicious Users: truly talented crackers (the advanced persistent threat) are rare, and if targeted "you will almost certainly be breached"; consult a security reference. Most are "script kiddies", dangerous because of numbers. He covers only the intersection of security and stability, chiefly the DDoS attack from a botnet (compromised PCs, increasingly Internet of Things devices). Attacks mostly go for applications rather than network gear and force you to saturate your outbound

bandwidth; session management is again the weak point. A specialised Circuit Breaker can limit damage by any one host (and guards against accidental floods). Vendor DDoS products must be configured and monitored, run in learning or baseline mode for at least a month.

- Remember This: users consume memory (minimise session contents, use the session only for caching so it can be purged); users do weird, random things (scripted tests are too predictable; look at fuzzing, property-based testing, simulation testing); malicious users exist (know your network design, make systems easy to patch, keep frameworks current); users gang up, so run special stress tests on deep links and hot URLs.

Blocked Threads

- Premise: managed runtimes (C#, Java, Ruby) almost never really crash, "The interpreter can be running, and the application can still be totally deadlocked". Multithreading brings scalability and concurrency errors, and the commonest failure is "navel-gazing": every thread waiting for Godot. Leaving the "fork, run, and die" model brings capacity at the price of a new risk.
- Central observation: most failures he has dealt with are not crashes but a process that runs and does nothing because every transaction thread is blocked "on some impossible outcome". The crash versus hung distinction matters only to engineers; the one variable that matters is "Is it generating revenue?" To users a system they can't use "might as well be a smoking crater".
- Hence he supplements internal monitors (log scraping, process and port monitoring) with external monitoring: a mock client outside the data center running synthetic transactions. Metrics like "successful logins" or "failed credit cards" reveal problems before an alert does.
- Where threads block: checking out from a connection pool, caches or object registries, external calls. Occasional blocking is normal if the code is by someone skilled in multithreading, "you are in a highly skilled minority". Four reasons it is hard: error permutations are too many to test; unexpected interactions break safe code; timing is crucial (hang probability rises with concurrent requests); developers never hit the application with 10,000 concurrent requests. So hangs are hard to find in development and cannot be tested out; craft code carefully with a small set of known primitives, preferably a proven library. He opposes rolling your own connection pool (reliable, safe pools are hard and exposing metrics makes it "a tedious grind").
- Design advice: if you synchronize methods on domain objects, rethink. It breaks when running on more than one server (in-memory coherence doesn't matter if another server changes the data) and threads that never block each other scale better. One elegant route: immutable domain objects for querying and rendering, with state changes issued as command objects ("Command Query Responsibility Separation").
- Spot the Blocking (see that sub-section for code): two innocent-looking lines (read a SKU parameter, call globalObjectCache.get) hide a blocking call; neither the call site nor the interface says so. A subclass can declare a method synchronized when its superclass or interface didn't, which violates the Liskov substitution principle; "Functional behavior composes, but concurrency does not compose."
- Case: the base GlobalObjectCache.get was a synchronized read-through registry, which worked because test cases returned quickly. A developer then subclassed it as RemoteAvailabilityCache, a textbook caching proxy with timestamped entries, to cache in-store availability queries to a remote inventory system (a few seconds each, valid fifteen minutes, nearly 25 percent of lookups on hot items, up to 4,000 concurrent requests). Functionally it was fine. Under stress the undersized inventory system, flooded by a busy front end, crashed, and every thread calling get() blocked because one thread was inside create() waiting for a response that never came: "Estragon and Vladimir, waiting endlessly for Godot."
- Moral: blocked threads, unbalanced capacities and missing timeouts at integration points (turning a layer failure into a cascading failure) combined to bring down the whole site. Business sponsors would laugh at "Should the site crash if it can't check availability"; architects and even the developer

would say it won't. "No one designed this failure mode into the combined system, but no one designed it out either."

- Use Caching, Carefully (sidebar): cap and configure cache memory (otherwise it starves memory and GC slows everything); monitor hit rates (a cache is a bet that generating once plus lookup costs less than generating every time); don't cache cheap things; build caches on weak references; give every cache an invalidation strategy that suits the number of instances, avoiding the Database Dogpile.
- Libraries: notorious sources of blocking, especially service-client libraries doing internal pooling with no configurable failure modes. With open source you may find and fix problems or search the issue log; with vendor code exercise it yourself (what happens when all connections are exhausted?). If it breaks easily, protect request threads: set timeouts if you can, otherwise wrap the call in a future with your own worker pool, abandoning the call on timeout, but "Here there be dragons" because you end up writing a reactive wrapper around the whole library. Also worth beating the vendor up for a better client library.
- Closing link: a blocked thread is often found near an integration point and can lead to chain reactions; "Blocked threads and slow responses can create a positive feedback loop, amplifying a minor problem into a total failure."
- Remember This: Blocked Threads is the proximate cause of most failures (including "gradual slowdown" and hung server) and leads to Chain Reactions and Cascading Failures; scrutinise resource pools (database deadlocks or bad exception handling can lose connections for ever); use proven primitives, not a home-made producer/consumer queue; defend with Timeouts ("You cannot prove that your code has no deadlocks in it, but you can make sure that no deadlock lasts forever"); beware code you cannot see, so test it and prefer open source.

Self-Denial Attacks

- Definition: "A self-denial attack describes any situation in which the system—or the extended system that includes humans—conspires against itself."
- Classic: the marketing email to a "select group of users" with a privileged offer. An offer for 10,000 users "is guaranteed to attract millions" as bargain hunters share coupon codes in milliseconds.
- Case: an Xbox 360 preorder email from a major electronics retailer gave the exact opening time and, via a deep link that accidentally bypassed Akamai, pulled every image, script and stylesheet from the origin servers. One minute before the opening, the site "lit up like a nova, then went dark. It was gone in sixty seconds." Others: reused coupon codes, pricing errors, a channel partner requesting every URL to cache pages. Paul Lord: "Good marketing can kill you at any time."
- Machine-made self-denial: in an ATG-based horizontal layer, one lock manager serves distributed cache coherency (acquire lock, update, release, broadcast invalidation). It is a singular resource that becomes bottleneck and then risk: a popular item modified by mistake leaves thousands of threads on hundreds of servers serialized on one write lock.
- Avoiding Self-Denial: build "shared-nothing" (each server runs without knowing of any other; an ideal approximated in practice). Where impractical, use decoupling middleware, make the shared resource horizontally scalable with backside synchronization, or design a fallback mode (e.g. optimistic locking if the pessimistic lock manager is unavailable). With warning and hardware load balancing, set aside or provision capacity for the promotion so that if it melts down the rest stays up. Autoscaling suffers from lag (minutes), so "pre-autoscale" before the marketing event. For human-facilitated attacks the keys are "training, education, and communication": keeping lines open may let you protect the system and still help marketing reach its goals.
- Remember This: keep lines of communication open (no mass emails with deep links, send in waves, create static "landing zone" pages for the first click, watch for session IDs embedded in URLs); protect shared resources (watch for Fight Club bugs, where increased front-end load causes exponentially increasing back-end processing); expect rapid redistribution of any cool offer, because "There's no such thing as limited distribution".

Scaling Effects

- Analogy: the square-cube law explains why there are no elephant-sized spiders (weight grows as n cubed, leg strength as n squared).
- Software: any many-to-one or many-to-few relationship can be hit when one side grows (a database fine with ten machines may crash when fifty more are added). Development has one machine and QA one or two, but production may be small, medium, large or humongous, so scaling effects are hard to see.
- Point-to-Point Communications: fine with one or two instances, but if every instance talks to every other, connections go as the square of the number of instances (O(n^2)), painful at a hundred. This might be fine for your eventual size, but know before production. Distinguish point-to-point inside a service from the normal fan-in between services (via a load balancer), which is a different case. Unless you are Microsoft or Google you can't build a test farm the size of production, so "This type of defect cannot be tested out; it must be designed out."
- No best choice: with only two servers point-to-point is fine (if it won't block when the other dies). As servers grow, replace it with UDP broadcasts (not bandwidth-efficient, load servers that don't care), multicast (only interested servers), publish/subscribe (picks up messages even if not listening, at infrastructure cost) or message queues, applying the XP principle "Do the simplest thing that will work".
- Shared Resources: commonly a "common services" project, cluster manager or lock manager that all members of a horizontal layer use; when overloaded it becomes a bottleneck limiting capacity. If it is redundant and nonexclusive, adding more scales the bottleneck. The most scalable architecture is shared-nothing, where capacity scales roughly linearly, but it may cost failover. Session failover needs coordination (session backup server, database table, a designated peer), "that implies some level of shared resources". You can approximate shared-nothing by reducing fan-in, e.g. pairs of app servers that are failover for each other.
- Worst case: the shared resource allocated for exclusive use while a client works. Contention then scales with transactions and clients; saturation creates a backlog, a backlog beyond the listen queue means failed transactions, and for cache managers that means stale data or "worse—loss of data integrity".
- Remember This: compare production and QA to spot scaling effects (what works at one-to-one may fail at production size); point-to-point connections grow as the square, so with tens of servers move to one-to-many; shared resources are bottlenecks and threats: stress-test them and be sure clients keep working if they slow or lock up.

Unbalanced Capacities

- Whether resources take months, weeks or seconds to provision, ratios between layers can mismatch so that one tier or service floods another beyond its capacity, especially with rate-limited or throttled APIs.
- Worked example: a front end with 3,000 request-handling threads (75 instances on 20 hosts) and a back-end scheduling service with 450 threads (6 hosts). Mostly the threads serve catalog and search, a few are in checkout, and a tiny fraction query the scheduling service to see whether a local team can install an item. You can compute from statistics and assumptions (notoriously easy to manipulate) how many calls arrive simultaneously, and size the service for it. But "Not necessarily."
- A self-denial attack, a free-installation offer for big appliances for one day, makes two, four or ten times as many calls. "The front end always has the ability to overwhelm the back end, because their capacities are not balanced."
- Building every service to the same size would be a gross misuse of capital: "99 percent idle except for one day out of five years". So build callers and providers resilient to a tsunami of requests: Circuit Breaker for callers; Handshaking and Backpressure so providers tell callers to throttle back; Bulkheads to reserve capacity for high-priority callers of critical services.

- Drive Out Through Testing: QA usually runs two servers per system, a one-to-one ratio, while production might be ten to one. QA can't be an exact replica of the enterprise, but a test harness that mimics a back end wilting under load verifies that the front end degrades gracefully (see Test Harnesses).
- Service providers face the other side: they expect tomorrow's workload to match yesterday's, but marketing campaigns, publicity, new front-end releases, and social media and link aggregator links change it, and they are removed from the people causing it. So: (1) capacity modelling to be in the ballpark ("Three thousand threads calling into seventy-five threads is not in the ballpark"); (2) test beyond usual workloads: double the calls the front end could possibly make, all on your most expensive transaction. A resilient system may slow or fail fast (see Fail Fast) but recovers after the load drops; crashes, hung threads, empty or nonsense responses signal a cascading failure; (3) use autoscaling if you can, but it has lag, can pass the problem to an overloaded platform service, and needs a financial constraint as risk management.
- Remember This: examine server and thread counts in QA versus production; Unbalanced Capacities is a special case of Scaling Effects (watch seasonal, market- or publicity-driven traffic changes, e.g. a hot Reddit post); virtualize QA and scale it up, trying different caller-to-provider ratios via automation; stress both sides of the interface: ten times the highest-ever demand on the most expensive transaction for providers, back-end calls that stop responding or get slow for front ends.

Dogpile

- Analogy: a large-scale power outage behaves like a software failure: a small event (a line grounding on a tree) becomes a cascading failure under high stress. Restoring power used to fail repeatedly: current demand from millions of air conditioners and refrigerators would overload the newly restored supply and trip breakers again. Modern appliances and controls have mitigated it, but two lessons remain.
- First lesson: only the fully assembled system exhibits that behaviour, no smaller subset of components can produce it, "Troubling when you think about QA environments". Second: steady-state load can differ greatly from startup or periodic load. A farm of booting app servers each connects to the database, loads seed data and starts with a cold cache, so most requests become queries and the database's transient load is much higher than after warm-up.
- Definition: "When a bunch of servers impose this transient load all at once, it's called a dogpile." It occurs when booting several servers (e.g. after a code upgrade), when a cron job triggers at midnight or on the hour, and when configuration management pushes a change (some tools offer a random slew).
- It also arises from external synchronized pulses, as people crossing at street corners clump and are resynchronized by the next light. Look for places where many threads block on one; when the logjam breaks, the freed threads dogpile downstream systems. Load tests can pulse if virtual user scripts have fixed-time waits; apply a small random delta to every pause.
- Remember This: dogpiles force you to pay for higher peak capacity than you would need if you spread the surge; use random clock slew (don't put all cron jobs on midnight or the hour); use increasing backoff times rather than a fixed retry interval, so callers are at different points in their backoff.

Force Multiplier

- Opening: automation is like a lever, allowing large movements with less effort: "It's a force multiplier."
- Outage Amplification: Reddit was down about ninety minutes with another ninety degraded on 11 August 2016. admins shut down the autoscaler to upgrade a ZooKeeper cluster; the package management system detected the autoscaler off and restarted it; it read partially migrated ZooKeeper data reflecting a much smaller environment; it decided too many servers were running and shut down many app and cache servers (start of the downtime); admins found the culprit,

overrode it and restored instances manually, but with empty caches they all hit the database at once, a dogpile that left Reddit up but unusably slow; finally caches warmed.

- His reading: the outage "emerged from a conflict between the automation platform's belief about the expected state of the system and the administrator's belief". The package manager couldn't know the autoscaler was meant to be down, and the autoscaler couldn't know that its source of truth was temporarily unable to report the truth: "Like HAL 9000".
- Service discovery: a distributed system reporting on other distributed systems. Nodes gossip to sync their knowledge and run health checks, removing nodes that stop responding. If node 3 is partitioned from the network, it can't tell "the rest of the universe just disappeared" from "I've got a blindfold on", and through gossip it spreads its belief to the cluster, so discovery reports zero services: "a smoking crater". A platform management service with the same belief would start a copy of every service.
- This arises mostly in "control plane" software (logging, monitoring, schedulers, scalers, load balancers, configuration management), which is not simply enacting an administrator's will but, like industrial robotics, senses current state, compares it with desired state and effects changes. Failures: desired state wrong (ZooKeeper), current state unsensed (partitioned node), or desired state computed incorrectly (a naive scheduler trying to drain a queue in fixed time could ask for infinity instances).
- Controls and Safeguards: borrowing OSHA's robot safety advice (limits on how far a joint rotates, rate limits, detecting unexpected resistance). Control plane equivalents: if more than 80 percent of the system seems unavailable, suspect the observer; apply hysteresis (see Governor) and start machines quickly but shut down slowly; when expected and observed state differ greatly, signal for confirmation (the "big yellow rotating warning lamp"); make resource consumers stateful enough to detect trying to spin up infinity instances; build in deceleration zones for momentum (if load is sensed every second but a VM takes five minutes to start, don't start 300 machines).
- Remember This: ask for help before causing havoc (build limiters and safeguards into infrastructure tools); beware of lag time and momentum (automated actions take longer than a monitoring interval); beware of illusions and superstitions (control systems compute an expected state and a belief about the current state, either of which may be wrong).

Slow Responses

- Restated from Socket-Based Protocols: a slow response is worse than a refused connection or error, particularly for middle-layer services. A quick failure lets the caller finish the transaction fast (success or failure per its logic); a slow response "ties up resources in the calling system and the called system".
- Causes: usually excessive demand (all request handlers busy, no slack); a symptom of an underlying problem such as a memory leak (high CPU that is really garbage collection); network congestion (rare in a LAN, possible across a WAN with a chatty protocol); and TCP stalls from hand-rolled socket protocols whose read() doesn't loop until the receive buffer is drained. They propagate layer to layer "in a gradual form of cascading failure".
- Remedy idea: let the system monitor its own performance against its service-level agreement. If a provider must answer within one hundred milliseconds and a moving average over twenty transactions exceeds it, start refusing requests, either at the application layer (error in the protocol) or at the connection layer (refusing new connections); such refusal must be documented and expected by callers.
- Remember This: Slow Responses trigger Cascading Failures when upstream timeouts are exceeded; for websites they cause more traffic as users hit Reload; consider Fail Fast (immediate error when average response time exceeds the allowed time, or at least the caller's timeout); hunt for memory leaks and resource contention, since contention for database connections produces Slow Responses, which aggravate the contention in a self-reinforcing cycle.

Unbounded Result Sets

- Principle: “Design with skepticism, and you will achieve resilience. Ask, What can system X do to hurt me?” The usual code sends a query and loops over the result set adding each row to a collection; if the database suddenly returns five million rows instead of a hundred, the application can exhaust memory or spin long after the user loses interest. Most applications trust their database server too much.
- Case, Black Monday: every instance in a farm of 100+ load-balanced commerce server instances began burning 100 percent CPU and then crashing with a HotSpot memory error minutes later, restarts taking minutes to warm caches (some crashed before finishing), so no more than 25 percent of capacity stayed up, on a 5 a.m. conference call of twenty people. Diagnosis in order: DoS suspected but crashes occurred before accepting requests; the CPU looked like garbage collection; “heap available” fell to zero before the JVM crash; a JVM crash instead of OutOfMemoryError implied native code, i.e. the type 2 JDBC driver, and the stack dump confirmed it; a DBA traced queries, all innocuous, the last one on the message table used for the database-backed JMS that tells instances to flush caches. That table should never exceed 1,000 rows but held more than ten million, and each instance selected all rows for update, locking them, built objects until out of memory and crashed; the rollback released the lock for the next instance to “step off the cliff”. Cause: no LIMIT clause. By the time they stabilised, Monday was Tuesday. (Why the table was so large is, he says, a different story.)
- Generalising: this occurs querying databases, calling services, and front-end apps calling APIs. Development data sets are small, so a year in production even “fetch customer’s orders” may return huge sets, treating the best customers “to the very worst performance”. “An unbounded result set occurs when the caller allows the other system to dictate terms. It’s a failure in handshaking.” The caller should always say how much of a response it will accept (TCP’s window field; search engine APIs with result counts and offsets). SQL has no standard limit syntax, and ORMs limit queries but usually not association traversal, so beware relationships that can accumulate unlimited children (orders to order lines, profiles to site visits, audit trails).
- Data shapes change between QA and production: early social media assumed bell-curve connections per user, but it is a power law, so an entity with a million times the average is guaranteed. (See the section for the per-database LIMIT/TOP/rownum recipes.) Breaking out of the loop after a maximum row count is incomplete, better than nothing, protecting the app server but wasting database capacity. Unbounded result sets are a common cause of slow responses and can result from violating steady state.
- Remember This: use realistic data volumes (production-sized test data also improves performance testing); paginate at the front end (request carries first item and count, reply says roughly how many results); don’t rely on data producers (“The only sensible numbers are zero, one, and lots”, and sooner or later they go berserk); put limits into other application-level protocols (service calls, RMI, DCOM, XML-RPC) too.

Wrapping Up

- The chapter covered “a lot of dark territory”: many ways systems are threatened internally and externally, antipatterns “found in nearly every service and application”.
- Now “It’s time to emerge from this vale of shadows into the light” and talk about the stability patterns that protect your software.

How this chapter builds the author’s case. It opens by saying that systems have outgrown old reliability thinking (billion-user scale, tight coupling, hidden linkages), so cracks are inevitable, and by promising that the next chapter’s patterns answer the antipatterns. The sections then chain: Integration Points are where cracks start, Chain Reactions and Cascading Failures spread them within and across layers, Blocked Threads are the usual mechanism, and Users, Self-Denial Attacks, Scaling Effects, Unbalanced Capacities, Dogpile and Force Multiplier show the stress that triggers or multiplies them, ending with Slow Responses and Unbounded Result Sets as symptoms of poor handshaking. He

leans on earlier cases and cross-refers constantly (Socket-Based Protocols to Slow Responses, Unbalanced Capacities in the availability-cache case, Self-Denial in Unbalanced Capacities, Dogpile in the Reddit case), and each Remember This names the remedy that later chapters develop: Circuit Breaker, Timeouts, Bulkheads, Decoupling Middleware, Handshaking, Fail Fast, Test Harnesses, Steady State, Governor, plus Chapter 11 for response handling and security.

CHAPTER 5

Stability Patterns

Place in the argument — why this chapter, why this title. The title names the flip side of the previous chapter. After the “vale of shadows” of the antipatterns, “it is time to come in to the light”: these patterns are “the inverse of the killers from the last chapter” and give “architecture and design guidance to reduce, eliminate, or mitigate the effects of cracks in the system”. So it sits right after the antipattern catalog and before Part II on design for production. The text does not argue the section order; the visible logic is that Timeouts, Circuit Breaker and Bulkheads answer Integration Points, Blocked Threads and Cascading Failures directly, while the later ones (Decoupling Middleware, Shed Load, Create Back Pressure, Governor) work at architecture and load level. Wrapping Up closes Part I and points to Part II.

Chapter opening — untitled section before the first sub-heading in the book

- Counterpart of the last chapter: there the antipatterns to avoid, here the patterns that mitigate cracks in the system. “Not one of these will help your software pass QA”, but they let you get a full night’s sleep once the software launches.
- Warning: do not assume more patterns means a superior system. “Count of patterns applied” is never a good quality metric.
- Instead develop “a recovery-oriented mind-set”: expect failures, and apply the patterns wisely to reduce the damage done by an individual failure.

Timeouts

- Every system is now distributed and networks are fallible: wire, switch, router or remote computer can break at any time. Code cannot wait forever for a response that might never come. “Hope is not a design method.”
- A timeout lets you stop waiting once you think the answer won’t come. He learned this porting BSD sockets to a mainframe UNIX: the networking code was riddled with error handling for different flavors of timeouts.
- Well-placed timeouts give fault isolation: another service’s problem need not become yours. But at higher abstraction levels good timeouts get rarer; many APIs offer a no-timeout call that he would rather see labeled “CheckoutAndMaybeKillMySystem”.
- Also inside one service: any resource pool can be exhausted, so a pool that blocks threads must have a timeout so callers eventually unblock (see Blocked Threads, page 62). Use the timeout form of mutexes and synchronization.
- To manage pervasive timeouts, wrap long-running operations in reusable primitives: e.g. checkout a database connection, query, build objects, return the connection (three points could hang). Use a query object for the part that changing and a generic gateway as template for connection handling, error handling, query execution and result processing, so it is right in one place; this also eases Circuit Breaker.
- Sidebar: this adds complexity, perhaps half the code, but aiming for production means handling “the slings and arrows of outrageous fortune”.
- Retries: immediate retry is likely to fail again, since within a data center problems “tend to last for a while”.
- To the client, being made to wait is bad: return an answer (failure, success, or a note that work is queued). Retrying while I wait may push you past my timeout and keeps my resources busy.
- Queuing for a slow retry is good, as with email store-and-forward: work is not lost and the system recovers when the remote server does. “Fast enough” behind a web API is probably 10 to 100 milliseconds.

- A circuit breaker can tabulate timeouts and trip. Versus Fail Fast (see Fail Fast): Timeouts protect you from someone else's failure and apply to outbound requests; Fail Fast reports why you won't process a transaction and applies to incoming ones. "They're two sides of the same coin." Timeouts on unbounded result sets are only a stopgap.
- Remember This: timeouts keep calls to Integration Points from becoming Blocked Threads, thus averting Cascading Failures; use them to recover from unexpected failures when "we just need to give up and keep moving"; consider delayed retries: most of the time, queue the operation and retry it later.

Circuit Breaker

- Analogy: house wiring overloaded by hidden coupling between gizmos heated and burned. Fuses, "designed to fail first", helped but were one-time use and penny-sized, so people made homemade copper "fuses". Circuit breakers: detect excess usage, fail first, open the circuit, and reset once danger has passed, so one subsystem fails without destroying the whole system.
- In software, wrap dangerous operations with a component that can circumvent calls when the system is unhealthy. Unlike retries, breakers "prevent operations rather than reexecute them".
- Closed: executes normally and counts failures. When failures (number or frequency) exceed a threshold it trips and opens. Open: calls fail immediately. After a suitable time it goes half-open: the next call is allowed; success resets to closed, failure reopens until another timeout elapses (see the state figure).
- When open: fail immediately, preferably with an exception different from an ordinary timeout, or use a fallback (last good or cached response, generic answer, secondary service).
- Breakers automatically degrade functionality under stress, with business impact, so involve stakeholders: should a retail system accept an order if it can't confirm availability, credit card or shipping address? Discussing the breaker can be more effective than asking for a requirements document.
- "Too many failures"? Five faults over five hours differ from five in thirty seconds, so fault density beats total count. He likes the Leaky Bucket pattern: increment a counter per fault, a timer decrements it toward zero, and a threshold shows faults arriving quickly.
- Operations is a stakeholder: log state changes, expose state, allow manual trip or reset. State-change frequency is "a leading indicator of problems elsewhere in the enterprise".
- Scope: a single process, shared by its threads but not across processes. Each instance rediscovers the failure, but sharing state adds out-of-process communication, so "the safety mechanism would introduce a new failure mode!" Avoid accidentally single-threading calls; start from an open source library.
- Remember This: "Don't do it if it hurts." It is the fundamental protection against Integration Points problems. Use together with Timeouts (the timeout indicates a problem, the breaker avoids calls). Expose, track and report state changes: a popped breaker is always abnormal and should be reported, recorded, trended and correlated.

Bulkheads

- Ship bulkheads create watertight compartments so one hull penetration does not sink the ship: "damage containment". Partition systems so a failure in one part doesn't destroy everything.
- Physical redundancy is the most common form (independent servers; two instances on one server, unless an external influence hits both).
- Largest scale: independent server farms, some reserved for critical applications, e.g. dedicated check-in servers unaffected by shared "flight status" queries in severe weather. That would have let the airline of Chapter 2 keep checking in passengers even if channel partners could not look up fares. In the cloud, use zones and regions; with functions as a service each invocation has its own compartment.

- Hidden coupling (see the figures): Foo and Bar both use Baz. If Foo is crushed by load or triggers a bug in Baz, Bar suffers, diagnosis is hard, and maintenance windows need both. For critical systems with strict SLAs, partition Baz into Pool 1 and Pool 2. They probably still share a database, “but that’s another antipattern”.
- Preserving all capabilities would be better, but assuming failure you must minimize damage. No single rule: examine the business impact of each lost capability against the architecture and find natural boundaries, technically feasible and financially beneficial, aligned with callers, functionality or topology.
- In the cloud bulkheads need not be permanent: carve out a VM cluster for one consumer.
- Smaller scales: binding a process to cores stops a berserk process draining the host. Within a process, separate thread groups, e.g. a reserved pool for administrative requests so a hung server can still be asked to shut down.
- Bulkheads maintain full or partial service in failure, especially in SOA, where each service is in effect a single point of failure for the enterprise.
- Remember This: “Save part of the ship.” Pick a useful granularity (thread pools, CPUs, servers). Consider them particularly with shared services: if a Chain Reaction takes your service down and the entire company halts, add Bulkheads.

Steady State

- Fiddling (“To handle something idly, ignorantly, or destructively”, per Roget) ends in the “ohnosecond”. Example: an engineer mistyped a resilver command and synced the good root disk from the new empty drive, destroying the operating system.
- Keep people off production. Needing “crank-turning and hand-holding” means servers are “pets” rather than “cattle” and leads to fiddling. A system should run at least one release cycle without human intervention; the extreme is immutable infrastructure (see Automated Deployments, page 242). Easy for a continuously deployed microservice, tough for a quarterly release.
- Principle: anything that accumulates resources (logs, rows, caches) is like a calculus bucket; it must drain at least as fast as it fills or overflow. “The Steady State pattern says that for every mechanism that accumulates a resource, some other mechanism must recycle that resource.”
- Data Purging: it “always gets the short end of the stick” because it doesn’t demo. Symptoms of growth: rising I/O rates on database servers, rising latency at constant load. Purging is detail work: referential integrity (orphaned rows) is half the battle, ensuring applications still work without the data the other half. RDBMS plus ORM handle dangling references badly; a document database won’t notice. So it is left for “six months after launch”, but at first launch “a fuse is lit”.
- Log Files: unbounded logs fill the filesystem; UNIX reserves the last 5 to 10 percent for root so applications see I/O errors at 90 to 95 percent full. Outcomes range from users noticing nothing, to a polite error, to a stack trace. Worst he saw: a reentrant exception handler that logged its own logging exceptions; once the disk filled it consumed eight CPUs, then all memory, then crashed.
- Fix: log rotation takes minutes (logrotate for legacy or third-party code). Rotate and purge all logs “or you’ll eventually spend time fixing the tool that’s supposed to help you fix the system”.
- Sidebar, compliance requires keeping logs for years, which single machines cannot. Get logs off production quickly, store them centrally (e.g. Logstash) and monitor for tampering.
- In-Memory Caching: “memory is like oxygen”. Ask: is the space of possible keys finite or infinite? Do cached items ever change? With unbounded keys, enforce size limits and invalidation; a periodic flush will usually do. Bad caching is the major cause of memory leaks and daily restarts, which keep administrators logged in.
- Remember This: avoid fiddling (run a deployment cycle with no manual cleanups or nightly restarts); purge data with application logic, since DBAs don’t know how the application behaves without the

data, especially with an ORM; limit caching ("speeds up applications, until it slows them down"); roll logs by size and keep compliance copies off production.

Fail Fast

- "If slow responses are worse than no response, the worst must surely be a slow failure response", like the DMV line. If the system can tell in advance it will fail, fail fast so the caller can move on.
- How: a large class of "resource unavailable" failures. A load balancer with no working server should refuse at once; queuing the request violates Fail Fast. A service can tell from the request which connections and integration points it needs, check them out and verify circuit breakers first, "the chef's mise en place". It can also check parameters in the controller before touching the database.
- Case, "We Got the Fax—It's All Black": for a photography company, old rendering software "rendered" all-zero black images when profiles, images, backgrounds or alpha masks were missing. The black prints wasted paper, chemicals and time, then came back for human diagnosis and expedited remakes that interrupted the pipeline. The team's renderer checked every font, image, background and mask on job arrival, preallocated memory, and reported failure to job control before wasting compute; broken orders left the pipeline. Software-induced remakes dropped to zero.
- The exception: disk space for the final image was not preallocated, at the customer's direction, because his "rock-solid" purging process was one guy deleting files by hand. Within a year the drives filled and the renderer failed after minutes of work.
- Report system failure (resources unavailable) differently from application failure (bad parameters), or an upstream breaker may trip because a user entered bad data and hit Reload several times.
- Remember This: inform callers quickly if you cannot meet your SLA, because otherwise "That just makes your problem into their problem". Reserve resources, verify Integration Points early ("don't do useless work"). Use for input validation before reserving resources.

Let It Crash

- "Sometimes the best thing you can do to create system-level stability is to abandon component-level stability." From Erlang. Chapter 2 showed not every error can be prevented; the state space exponentiates; assume errors happen.
- Usually we recover with handlers and try-finally. But "The cleanest state your program can ever have is right after startup", and error recovery is hard and unreliable, so get back to clean startup quickly. It needs four things.
- Limited Granularity: a boundary so a component crashes in isolation. In Erlang or Elixir it is the actor; Akka overlays actors on other runtimes, with more code review to enforce its rules. In microservices, a whole instance may be right.
- Fast Replacement: otherwise many restarting instances degrade performance, or in the limit all are busy restarting. Instances depend on stack: Go container in milliseconds, crash the container; NodeJS on a long-running VM, crash only the process (a new VM takes minutes); an aging JavaEE application with minutes of startup: "Let it crash" is not the right strategy.
- Supervision: a bash while loop becomes a fork bomb if the problem persists. Actor systems use a hierarchical supervisor tree; a supervisor restarts the child, restarts all its children, or crashes itself, passing up to its own supervisor. Tree design is integral to system design. "The supervisor is not the service consumer"; managing a worker differs from requesting work. Supervisors track restart density and may crash themselves, since dense restarts mean unclean state or a whole system in jeopardy and a supervisor masking it. PaaS and autoscalers lack discretion, so they are not supervisors.
- Reintegration: callers' circuit breakers reintegrate a directly called instance; a pooled instance must rejoin the pool (PaaS handles containers; static VMs when load balancer health checks pass).

- Remember This: crash components to save systems, though counterintuitive; restart fast and reintegrate automatically; isolate with Circuit Breakers and supervisors so crashes don't touch unrelated functionality; don't crash monoliths.

Handshaking

- Handshaking is signaling between devices that regulate communication: serial receivers signaling readiness, modem negotiation, TCP's three-phase handshake (page 37) which also lets a receiver halt the sender. Ubiquitous at low level, "almost nonexistent at the application level".
- "The sad truth is that HTTP isn't good at shaking hands." 503 Service Unavailable means a temporary condition, but most clients treat anything besides 200, 403 or 302 as fatal. RPC technologies (CORBA, DCOM, RMI) are equally bad at signaling readiness.
- It lets the server protect itself by throttling its own workload and rejecting incoming work. Closest with HTTP: the web server answers the load balancer's periodic health check with 503 or an error page, so no more work is sent. This helps only for web services and fails if all servers are busy. With several services, each can offer a health check query.
- Most valuable when unbalanced capacities cause slow responses: a server that cannot meet its SLAs should ask the caller to back off. Behind a load balancer it has only on/off control, "a crude mechanism". Best bet: build handshaking into custom protocols.
- Circuit Breaker is the stopgap for services that can't handshake: you make the call and track whether it works.
- Remember This: create cooperative demand control (both ends must be built for it); consider health checks as handshaking with the load balancer; build handshaking into your own socket protocols.

Test Harnesses

- Distributed failure modes are hard to provoke in QA, so we use an integration environment. Problems: which version to test against? Testing against release versions would, "by induction", limit the company to one new piece of software at a time. Interdependencies make the environment "unitary", one global test duplicating production and needing equally rigorous change control.
- Deeper: such environments verify only behavior while dependencies work. Even a forced error code (error 14916 unless the request includes the date of the last telephone sanitization) is the remote system within spec. But the book's theme is that every system will eventually operate out of spec, so we must test when the remote system "goes wonky".
- So build test harnesses emulating the remote end of each integration point, preserving isolation. "A good test harness should be devious": as nasty as real systems, leaving scars so the system under test turns cynical.
- Sidebar, Why Not Mock Objects?: a mock can only be trained to produce behavior conforming to the defined interface; a harness is a separate server not obliged to conform to any interface.
- A web-service harness should refuse connections, leave them in the listen queue, send only RESETs, connect but send nothing, never acknowledge, send headers but no body, one byte every thirty seconds, HTML instead of XML, or refuse all credentials. These fall into transport, network protocol, application protocol and application logic problems.
- Not via flags in the application: costly, and risky if promoted to production. Integration environments examine only layer seven, and not all of that. A harness knows it is for testing, can use low-level network APIs, and should "act like a little hacker".
- Trick: different ports mean different misbehavior (10200 accepts, never replies; 10201 replies with /dev/random; 10202 drops at once), so one harness breaks many applications and developers can run their own. Log requests in case the application dies silently.
- Fault injection, latency and packet reordering unearth hidden dependencies: "The only limit is your imagination." It leads toward chaos engineering (Chapter 17).

- Remember This: emulate out-of-spec failures; stress the caller with slow, no or garbage responses; share harnesses via a killer server on several ports; supplement, don't replace, unit, acceptance and penetration tests, since a harness checks "nonfunctional" behavior with isolation.

Decoupling Middleware

- Middleware is a "graceless name" for tools that integrate systems never meant to work together, the "connective tissue" between islands of automation. It is inherently messy; SOA now takes the attention.
- Done well it integrates and decouples (systems drop specific knowledge of each other). "Since integration points are the number one cause of instability, this looks like a good thing."
- Synchronous call-and-response (RPC, HTTP, RMI, CORBA, "JSON over HTTP, too") forces the caller to wait and both sides to be active at once. "Tightly coupled middleware amplifies shocks"; synchronous calls "facilitate cascading failures".
- Looser forms (MQseries, queues, publish/subscribe, SMTP, SMS) let systems process messages at different times and places. Message-oriented middleware decouples in space and time; since the requester doesn't wait, it "cannot produce a cascading failure".
- Trade-off: synchronous is logically simple (credit card authorization by RPC lets the app decide next steps). Asynchronous must handle failed or unanswered requests, exception queues, late responses and callbacks, and involves business sponsors on acceptable financial risk. "Designing asynchronous processes is inherently harder."
- Unlike most patterns here, this has a switching cost: moving to asynchronous needs very different design.
- Remember This: "Decide at the last responsible moment", yet this architecture decision ripples everywhere and is nearly irreversible, so make it early. Total decoupling avoids Integration Points, Cascading Failures, Slow Responses and Blocked Threads problems and makes applications more adaptable. Learn many architectures; not every system should be three-tier with a relational database.

Shed Load

- Services and open APIs "have zero control over their demand". However strong the load balancers or fast the scaling, "the world can always make more load than you can handle".
- TCP's listen queue rejects new connections with a reset when full. But services often fall over earlier from contention for pooled resources: threads slow, extra threads use too much RAM and CPU, response times lengthen until callers time out. To an outside observer there's no difference between "really, really slow" and "down".
- Services should model TCP: when load is too high, refuse new requests (related to Fail Fast). Ideal measure: the service watches its own performance against its SLA. Otherwise a semaphore capping concurrent requests; a queue between accepting and processing costs complexity and latency. With a load balancer, a 503 on the health check page makes it back off.
- Inside a boundary, back pressure (see Create Back Pressure) is more efficient for balanced throughput; shed load as a secondary measure.
- Remember This: "You can't out-scale the world." Avoid slow responses ("Creating slow responses is being a bad citizen"). Use load balancers as shock absorbers: individual instances report 503 and balancers recycle connections quickly.

Create Back Pressure

- "Every performance problem starts with a queue backing up somewhere." An unbounded queue can consume all memory and, by Little's law, its response time heads to infinity. So bound queues.
- When full there are few options: pretend to accept but drop the item; accept it and drop something else; refuse it; block the producer. Dropping may suit data that loses value with age (drop the oldest).

- Blocking applies “back pressure” upstream, ideally to the ultimate client, slowed until the queue releases. TCP does this with window fields until socket writes block. “The mechanisms change but the idea is still to slow the producer down until the consumer can catch up.”
- Caveats: it can cause blocked threads, so distinguish a temporary condition from a broken consumer. Best with asynchronous calls. Helps only when the consumer pool is finite.
- Example: a create-tag API limited by storage and indexing slows progressively until it seems offline. Instead a blocking queue allows each API server 100 simultaneous storage calls; the 101st blocks. “That blocking is the back pressure.”
- A flat limit is crude; smarter is blocking at the receiver, with a wrapper service adjusting queue size.
- Yet a thread still waits, and blocked threads lead to downtime (page 62). At system edges they frustrate users or provoke retries, so back pressure works best within a boundary; at edges also use load shedding and asynchronous calls. Accept calls on one thread pool, call storage on another, so the handler can time out and answer HTTP 503, or queue the command for later indexing and answer HTTP 202.
- Inside the boundary, back pressure shows as a performance problem or timeouts and does indicate one: consumers collectively exceed the provider’s capacity, though a single consumer gone nuts (an attack of self-denial or organic change) may be responsible. Monitoring must know when it kicks in to tell fluctuation from trend.
- Remember This: it creates safety by slowing consumers (“The only alternative is to let them crash the provider”); apply it within a system boundary and use load shedding across boundaries; queues must be finite for response times to be finite; when full, all options are unpleasant (drop data, refuse work, block), so don’t block forever.

Governor

- Recall Reddit in Force Multiplier (page 80): its autoscaler read a partial configuration mid-ZooKeeper migration and shut down nearly every instance. The flip side: a job scheduler spinning up too many instances, and an invoice “written in scientific notation”.
- “Automation has no judgment.” It fails quickly, leaving recovery rather than intervention. Use automation for repetitive tasks and fast response, humans for perceiving the whole situation.
- Answer from eighteenth-century steam: engines run too fast break. A governor limits engine speed even if the power source could drive more. For software, slow the rate of actions; Reddit limited its autoscaler to shutting down a certain percentage of instances at a time.
- A governor is stateful and time-aware, and asymmetric: actions have a safe and an unsafe direction. Unsafe: shutting down instances, deleting data, blocking client IP addresses.
- Definitions of safe conflict: shutting down is unsafe for availability, spinning up for cost. They define a U-shaped curve, so actions are safe within a range (a thousand EC2 instances in budget, two thousand needs slowing). The curve is the governor’s response curve: fast inside the safe zone, increasing resistance outside.
- Point: slow things enough for humans to get involved, connected to monitoring to alert them and give visibility.
- Remember This: slow things down to allow intervention (humans are better at situational thinking); apply resistance in the unsafe direction (shutting down, deleting, blocking); consider a response curve.

Wrapping Up

- Unlikely combinations eventually occur: a small service at ten million requests a day for three years has 10,950,000,000 chances for something to go wrong, against four hundred billion stars in the Milky Way. “Astronomically unlikely coincidences happen all the time.”

- Failures are inevitable. Stability antipatterns amplify transient events and accelerate cracks; avoiding them does not prevent bad things but minimizes damage.
- Judiciously applied, patterns keep software up “come hell or high water”. The key is judgment: examine requirements cynically, view other enterprise systems with suspicion, identify threats and apply patterns appropriate to each. “Paranoia is good engineering.”
- Production no longer resembles a desktop or laptop (network, security restrictions, runtime limits), so Part II turns to design for production operations.

How this chapter builds the author’s case. It converts the previous chapter’s antipatterns into remedies and pairs them with it (Timeouts stop Integration Points becoming Blocked Threads and Cascading Failures; Bulkheads answer Chain Reaction; Steady State helps avoid Slow Responses). It reuses earlier material: the Chapter 2 airline, Reddit from Force Multiplier, Blocked Threads on page 62 and TCP on page 37. It carries the book’s theme that every system eventually operates out of spec and failure must be expected, ends on judgment and paranoia, and hands over to Part II, with Test Harnesses pointing ahead to Chapter 17 and Steady State to Automated Deployments on page 242.

PART II OVERVIEW

Part II — Design for Production

Author's description (Preface): after stability, "the next concern is ongoing operations": systems that run in complex virtualized, containerized, load-balanced environments; the Part gives patterns for control, transparency and availability.

The theme of the Part

- Design the system for the people and platforms that run it in production, working up from the physical substrate to systemic concerns.

How the chapters build the theme

- Chapter 6, Case Study: a holiday-season outage shows why visibility and control matter.
- Chapters 7 to 9, Foundations, Processes on Machines, Interconnect: the layers from hosts and networks, to instances as good citizens, to the connections between instances.
- Chapters 10 and 11, Control Plane and Security: managing the sea of instances as a system, then adding the adversary.

What the Part contributes to the book

- A system that can be operated, observed and defended; Part III is about changing it safely.

CHAPTER 6

Case Study: Phenomenal Cosmic Powers, Itty-Bitty Living Space

> **Place in the argument — why this chapter, why this title.** First chapter of Part II (Design for Production), and a case study, so a real outage comes before any design prescriptions. The title jokes about huge power in a tiny space: a big traffic surge funnelled into a scheduling system that handles only twenty-five concurrent requests. After the calendar opening and the holiday-season headings, the medical sub-headings (Taking the Pulse through Winding Down) follow one incident in order: observe, diagnose, escalate, choose a fix, test it, recover. The text gives no explicit reason for its placement, but ends on Recovery-Oriented Computing, "a major theme in this book".

Chapter opening — untitled section before the first sub-heading in the book

- Lilius's calendar fixed the Julian drift; the Gregorian error "is cyclic, not cumulative". Calendars mark holy days, but "No business in the world actually lives by the Gregorian calendar"; each industry has its own almanac (open enrollment, Valentine's Day).
- Retailers live by the "holiday season": up to 50 percent of annual revenue falls between November 1 and December 31. Thanksgiving starts it, shopper panic produces Black Friday, store traffic can quadruple and online traffic can increase by 1,000 percent. "This is the real load test, the only one that matters."

Baby's First Christmas

- The client launched a new online store in the summer; launching a site is like having a baby, full of horrifying discoveries.
- "Cautious optimism" rested on three things: nearly doubled servers; hard data from burst events (mispriced items) showing where latency climbed; and more visibility and control over the store internals, through the application server and tools built around it, than on any other system. That "would ultimately prove to be the difference".
- The SOC in another city normally watched nights and weekends, with local engineering as escalation backup. For the weekend the local team staffed the site around the clock; the author still packed his laptop for Thanksgiving three states away.

Taking the Pulse

- Load tests are usually black-box, with data from the load generators. The author had used the application server's HTML admin GUI to see latency, free heap, active threads and sessions. A GUI suits exploring, but is "downright impractical" for thirty or forty servers.
- So he wrote Perl modules that screen-scraped the GUI to get and set properties and invoke methods, and scripts that sampled all servers in a loop. After launch the team learned the site's normal rhythm from these stats.

> *"nothing beats the pattern-matching power of the human brain"*

Thanksgiving Day

- Thursday morning sessions already rivaled a normal week's peak; the DBA confirmed no double-submitted orders. By midnight orders equalled all of October, with latency stressed but steady. "It passed the first killer load test."

Black Friday

- Friday latency was about 250 milliseconds as expected. Then, far from his access point, the author got the SOC's call: SiteScope red on all DRPs, and rolling restarts failing immediately.
- SiteScope simulates customers, so red means customers can't shop. DRPs are the request-handling instances (Dynamo Request Protocol) of an ATG site. "All DRPs red" meant the site was down, losing about a million dollars an hour. Restarting takes about ten minutes per host, and more than four or five hosts at once slows the database. They were treading water but sinking.

Vital Signs

- Twenty minutes in: very high sessions, high but unlimited bandwidth, high latency, really low CPU on web, application and database, healthy search servers (the usual culprit), and request threads almost all busy, many for over five seconds.
- Latency was effectively infinite because requests timed out. “Response time is always a lagging indicator”: only finished requests are measured.
- Thread dumps; Ashok saw low CPU and busy threads on the back-end order management system too. Ninety minutes after the site went down, they neared missing the SLA.

Diagnostic Tests

- On every DRP a few threads were calling the back end and most were waiting for a connection, “blocked on a resource pool, one that had no timeout”. All 3,000 threads were tied up doing nothing, explaining the low CPU.
- In order management (450 threads), some threads were calling an external integration point for home delivery scheduling and all others were blocked behind them. A different group, without 24/7 staff, owns that system; they were paged.

Call In a Specialist

- Of four scheduling servers, two were down for weekend maintenance and one was malfunctioning. The sole survivor handled twenty-five concurrent requests before hanging, and was probably getting ninety. It was at 100 percent CPU, but its on-call engineer ignored the pages: “All the false positives had quite effectively trained them to ignore high CPU conditions.”
- The sponsor then disclosed marketing’s Friday insert offering free home delivery on orders before Monday. “The entire line … went silent”.
- The chain: 3,000 store threads on 100 servers swamp order management (450 threads shared between front-end requests and order processing), which swamps scheduling (twenty-five). “There’s no playbook for this situation”, so they had to improvise.

Compare Treatment Options

- Proposals failed because the code’s behavior under those conditions was unknown. The only answer was to throttle schedule-availability calls, which order management could not do.
- The store’s code had a separate resource-pool subclass just for scheduling connections (perhaps Conway’s law). That could be the throttle. With no enabled property, a pool with a zero maximum is effectively disabled; the developers said a null instead of a connection would show users a polite message that scheduling was unavailable. “Good enough.”

Does the Condition Respond to Treatment?

- Setting max and checkoutBlockTime to zero on one DRP did nothing, since max has an effect only at pool start-up. Calling stopService() and startService() worked, but the load manager then sent every request to that one DRP, “crushed”. “But at least we had a strategy.”
- Sidebar, Recovery-Oriented Computing (Berkeley and Stanford): failures are inevitable; “A priori prediction of all failure modes is not possible”; human action is a major source of failures. ROC accepts that failures will happen and aims at survivability, “a major theme in this book”.
- The author ran the scripts on all DRPs. Restarting components, not entire servers, is “a key concept of recovery-oriented computing”: changing config files and restarting everything would have taken over six hours under that load; reconfiguring the pool took under five minutes (once they knew what to do). About ninety seconds after the scripts finished, the DRPs went green.

Winding Down

- The author wrote one script to set max, stop and restart the service, for use from the operations center or the “command post”. It was used constantly all weekend: raised at light load, set to one (not zero) at heavy load, since zero disabled home delivery.

How this chapter builds the author’s case. It gives the concrete failure the rest of Part II answers: a blocking call with no timeout, a pool, a small downstream system and an ignored alert. Its sidebar ties to the book’s theme that failures are inevitable, and the control built into the application server, including restarting a component, saved the weekend. The text names no later chapter.

CHAPTER 7

Foundations

Place in the argument — why this chapter, why this title. Opens the run of chapters on "design for production" after the case study in Chapter 6, and is the first and lowest layer. The title names the bottom layer of the author's layer diagram (Foundation: hardware, VMs, IP addresses, physical network). The contents order follows his stated plan: first clear up networks, hostnames and IP addresses; then the "code holders", physical hosts, virtual machines and containers; each kind of deployment has concerns that software designs must account for. Wrapping Up lists the questions common to every environment.

Chapter opening — untitled section before the first sub-heading in the book

- The previous chapter's outage was only recoverable because "we already had good visibility into the running system" and control over it; there was no time to add logging, and rebooting servers after every configuration change would have made recovery impossible.
- The next chapters cover those ingredients, leading to "design for production": "thinking about production issues as first-class concerns". That includes the production network (possibly very different from development), logging and monitoring, runtime control and security, and designing for the people who do operations. "Operators are users, too."

> *"If your system's front end is Disney World, then operators get to use the secret tunnels beneath the park."*

- The chapters work through layers, each zooming out to wider, more distributed concerns: Foundation (hardware, VMs, IP addresses, physical network); Instances; Interconnect; Control Plane; Operations (security, availability, capacity, status, communication). This chapter covers the Foundation.
- The words "as a service" appear nowhere in the diagram: the IaaS/PaaS distinctions "have broken down completely". More useful is to ask which layers a platform drives completely by API, which responsibilities move from operations to developers, and what stays an application-level concern.

Networking in the Data Center and the Cloud

- Data center and cloud networks have more redundancy and security than desktop networks, and with virtualization applications "can behave very differently than they do in the safe confines of the IDE".
- NICs and Names: a hostname has two meanings. One is the name the OS uses for itself (hostname plus default search domain gives the FQDN). The other is the external name that other computers resolve through DNS to an IP address. Nothing guarantees they match: a machine may be spock.example.com but be reached as mail.example.com and www.example.com.
- A machine identifies the whole machine; a DNS name identifies an IP address. "DNS name to IP address" is many-to-many, yet the machine acts as if it has one hostname. Many programs assume the self-assigned FQDN resolves back to itself, "largely true for development machines and largely untrue for production services".
- A second many-to-many: several NICs per machine, each on its own network with an IP address (multihoming). Dev boxes have several for mobility (Ethernet, Wi-Fi, loopback). Data center machines are multihomed to put administration and monitoring on a separate network and to segment high-volume traffic such as backups. An application unaware of this "will easily end up accepting connections from the wrong networks", such as administrative connections on the production network.
- In the example server with four interfaces, two are production NICs to different switches (high availability, load balanced or failover pair), so the machine has more than one name from outside. Bonding (teaming) shares one IP address; bonded interfaces on different switches need switch configuration "or else routing loops can result".

- The backup NIC keeps bursts of huge data off the production network (separate switches or VLANs). The administrative network lets services such as SSH be bound only there, protecting against a breached firewall.
- Programming for Multiple Networks: by default, a socket listens on every interface. The easy version opens a socket on every interface ("Bad news!"), so we must specify the address, as in net.Listen with spock.example.com:8080 instead of just :8080. On a multihomed machine getLocalHost() returns only the address of the internal hostname, so servers need configurable properties naming the interfaces to bind.
- Outbound Connections: needing to choose the outgoing interface is exceedingly rare and, in production, he would regard it as a host configuration error. The exception is two NICs on two switches bonded as bond0; the fix is a routing table with a default gateway using bond0.

Physical Hosts, Virtual Machines, and Containers

- All machines eventually run on silicon, but "A bewildering array of deployment options force us to think about the machines' identities and lifespans." A design that works in a physical data center "may cost too much or fail utterly in a containerized cloud environment".
- Physical Hosts: CPUs have converged with development boxes, but the story is "all about expendable hardware". Data center hardware used to be built for reliability of the individual box; now services are load-balanced across enough hosts that losing one is not catastrophic, so each host should be cheap. Exceptions are workloads needing large RAM (graph processing) and GPU computing.
- Storage is mostly SAN or NAS, not on the host. To an application both look like a mount point or drive letter; "Just measure the throughput". Design for production hardware "just means building to scale horizontally", shifting specialized workloads to their own boxes.
- Virtual Machines in the Data Center: virtualization promised developers a common hardware appearance and managers reined-in "server sprawl". The managers' story won. Downsides: unpredictable performance, as hosts are oversubscribed (16 cores hosting 32 allocated is 100 percent oversubscribed), giving contention and random slowdowns that a guest OS can hardly monitor.
- Applications must not be sensitive to loss or slowdown of any one host. Watch for: synchronous responses required from the whole cluster; "special" machines like cluster or lock managers unless another can take over without reconfiguration; subtle dependence on request or event ordering.
- VMs make clock problems worse: a VM can be suspended for an indefinite span or migrated to a host with skew, and the tools that resync the clock make it jump around more. "The bottom line is: don't trust the OS clock." Use an external source like a local NTP server.
- Containers in the Data Center: pushed by developers, they promise VM-like isolation with a developer-friendly build; the hypothesis is never asking whether production matches QA. A container has a short-lived identity, so it should not be configured per instance (old monitoring such as Nagios needs reconfiguring and bouncing for each change), and has little local storage.
- The hardest part is the network. By default, ports aren't exposed on the host. A common pattern is the overlay network: VLANs create a virtual network among containers with its own IP space and software-switch routing, managed by control plane software (see Virtual LANs for Virtual Machines). Second hardest is placing enough instances of the right types on the right machines. Containers start in milliseconds and come and go, so control software must be told the desired load out and know the geographic distribution of hosts. Kubernetes, Mesos and Docker Swarm are attacking networking and allocation; none is dominant.
- Design for containers: the image moves between environments, so it can't hold production credentials (a 12-factor app handles this; otherwise inject configuration, and look into password vaulting). Externalize networking too: no hostnames or port numbers in images, since links are established by the control plane at start-up (see The 12-Factor App). Avoid long start-up or warm-up;

"Aim for a total startup time of one second." Debugging inside a container is notoriously hard, so applications "need to send their telemetry out to a data collector".

- Virtual Machines in the Cloud: AWS dominates, and cloud-native systems have better availability and cost than "lift and shift". Any individual cloud VM has worse availability than a physical machine: an extra OS, start/stop by the control plane without notice, resource contention.
- Machine identity is ephemeral: a VM's IP changes on every boot, and keeping addresses in files means renting Elastic IPs, which are limited. The general rule is that VMs have to "volunteer" to do work: autoscaling and load balancers for HTTP, competing consumers on a queue for asynchronous load.
- Cloud VMs default to one private NIC, limited to about 64,000 connections by socket numbers (1 to 65535), so add production NICs for connections and another NIC for management. Don't expose SSH on every front-end NIC; use a single bastion or jumphost with strong SSH logging.
- Containers in the Cloud: they combine both sets of challenges; individual service design is much as in the data center, but the challenge is building containers into a whole system. "using containers pushes some complexity out of the boxes and into the control plane" (see Chapter 10, Control Plane).

Wrapping Up

- Cloud and platform-as-a-service "move the boundary of responsibility back and forth between application development, platform development, operations, and infrastructure", but some questions are common to every environment: How is the network structured, and will a machine have NICs on different networks with different jobs? Do machines have long-lasting identities? Are machines automatically set up and torn down, and how are their images managed?
- These answers never appear on a Kanban board or a Jira ticket, but are "essential to making a smooth transition to operations".

> **How this chapter builds the author's case.** It looks back to Chapter 6, where visibility and control over the running system made recovery possible, and turns that into "design for production" with operators as users. It starts the layer diagram at the Foundation. Forward, the author says the next chapter looks at how individual machine instances behave and how we control them, and containers' complexity moves into the control plane (Chapter 10).

CHAPTER 8

Processes on Machines

> **Place in the argument — why this chapter, why this title.** Chapter 7 covered the network and physical environments; this chapter focuses on "the individual instances", which must be "good citizens" by providing transparency, accepting control, handling configuration nicely and managing connections. The title names the unit: a process on a machine. Through his car analogy ("fuel, fire, and air" become "code, config, and connection") he orders the sections: Code, Configuration, Transparency; connection is left to Chapter 9. The link to the stability patterns of Chapter 5 is that each instance must "accept stress and insults with tolerance and grace".

Chapter opening — untitled section before the first sub-heading in the book

- Our vocabulary "hasn't really kept up with our technology": the word server may mean a VM on a physical host or an OS process, and a container process is also a process on the host. So he fixes terms, "at the risk of seeming hopelessly pedantic".
- **Service**: processes across machines that together deliver a unit of functionality (it may mix executables, such as application code plus a database, and may have one load-balanced IP address or several under one DNS name). **Instance**: an installation on one machine (container, virtual or physical) out of a load-balanced array of the same executable. **Executable**: an artifact created by a build process that a machine can launch as a process. **Process**: the runtime image of an executable. **Installation**: the executable plus its directories, configuration files and other resources on a machine. **Deployment**: the act of creating an installation; it "should be automated, with the deployment definition kept in source control."
- Deployment view ("Loan Request" example): sources become binaries and binaries become deployments, by moving inert files. The pipeline stages tag the build as passed, and the "very same tagged binary" is laid down on each machine.
- Runtime view: each machine runs an instance of the same binary, all behind an HAProxy load balancer whose address is bound to the DNS name loanrequest.example.com. "A lot of architectural confusion stems from attempts to cram both static and dynamic views into the same figure."
- Why precision matters: teams "have been bitten when different people use the same word for different things." Told to "reboot the server," you cannot be sure whether someone will kill one process or the whole machine.

Code

- The rules are "mostly about making sure that you know exactly what goes into the code on the instance." He wants a strong "chain of custody" from developer to production, so that no unauthorized party can sneak code in.
- Building the Code: use version control, for code only (it handles dependencies poorly). Builds download dependencies, by default from the Internet; that is "convenient but not safe", since a dependency can be silently replaced by a man-in-the-middle attack or a compromised upstream repository. Move to a private repository, admitting libraries only when signatures match the upstream provider's information. Build plugins too: a compromised Jenkins plugin was used to attack a company's enterprise customer.
- Never make production builds on developer machines, which are "hopelessly polluted." Build on a CI server that writes to a repository nobody else can write into.
- Immutable and Disposable Infrastructure: Chef, Puppet and Ansible apply changes to running machines (the "layers of stucco" approach), which has two problems. Side effects creep in that recipes do not describe: a package's post-install script changes TCP tuning parameters, a later RPM changes a subset, and the machine reaches a state "that cannot be re-created by either the original or the new recipes". And partial failures leave undefined states that convergence does not always fix.

- The DevOps and cloud answer is “immutable infrastructure”: start from a known base image, apply a fixed set of changes, never patch. For a container, “we don’t patch up the container; we just build a new one instead”, launch it and throw the old one away. The point is disposability: we can throw the environment away “piece by piece or as a whole, and start over.”

Configuration

- Properties (hostnames, ports, keys, passwords) must all be right, even “at 1 a.m. when Daylight Saving Time kicks in.” Configuration suffers from “hidden linkages and high complexity”, two big causes of operator error, and it is part of the system’s user interface for developers and operators.
- Configuration Files: keep per-machine properties apart from the rest “so nobody ever has to ask, ‘Are those supposed to be different?’” Binaries must not change per environment but properties must, so code looks outside the deployment directory. Production database passwords make these files the most sensitive, so keep them out of the source tree: “Sooner or later, you’ll accidentally commit a production password to version control.” Keep them in a separate, locked-down repository with controls on granting and revoking access.
- Configuration with Disposable Infrastructure: in image-based environments, files cannot change per instance and some instances live too briefly. Two approaches. Injection (environment variables, or a text blob such as EC2 user data) means the app must know its target environment. A configuration service (ZooKeeper, etcd) creates a hard dependency, so any downtime is immediately a Severity 1 problem because instances cannot start. These are complex distributed systems needing topology and capacity planning, and ZooKeeper “is scalable but not elastic”. Use them only within a broader organizational strategy: “Most small teams are better off using injected config”.
- Naming Configuration Properties: avoid “unforced errors.” Name by function, not nature: hostname does not say which host, like naming a variable integer because it is one. Call it authenticationProvider and the admin knows to look for an LDAP or Active Directory host.

Transparency

- Shipboard engineers detect trouble from engine sound and, knowing the linkages, find it fast. Our systems run in “invisible, faceless, far-distant boxes”, so “we must facilitate that awareness by building transparency into our systems.”
- **Transparency**: the qualities that let operators, developers and business sponsors understand historical trends, present conditions, instantaneous state and future projections. “Transparent systems communicate, and in communicating, they train their attendant humans.”
- Why: the Black Friday problem (Chapter 6) was solved with component-level visibility that “was no accident”; otherwise “like having a sick goldfish”. Transparent systems are easier to debug, so “transparent systems will mature faster than opaque ones.” Without trusted data, decisions follow political clout. Without transparency a system “cannot survive long in production”: administrators cannot tune it, developers cannot improve it, sponsors will not fund it, and it drifts into decay.
- This section is the instance-level slice; Chapter 10 (Control Plane) builds system-level transparency. “The job of an individual instance is to reveal enough data to enable those perspectives.”
- Designing for Transparency: “adding transparency” late is as effective as “adding quality.” “Strictly local visibility leads to strictly local optimization”: a retailer’s batch jobs finished two hours earlier but items still waited on a parallel process until 5 or 6 a.m. One server’s cache stats hid that every server invalidated all the others’ caches; seen on one page the problem was obvious, and adding servers would have worsened it. Keep monitoring like “an exoskeleton built around your system, not woven into it”: alert metrics, thresholds and health roll-ups stay outside the instance, as policy decisions change at a different rate than code.
- Enabling Technologies: a process is opaque, so the first trick is getting information out. Black-box technology sits outside the process and can be added by operations after delivery (a plain text log any scraper can read). White-box technology runs inside, as an agent library with an API, integrated during development; the coupling is “a small price to pay”.

- Logging: log files are still “the most reliable, versatile information vehicle”, and “Nothing is more loosely coupled than log files”. They are white-box but badly abused, so:
- Log Locations: not a logs directory under the install directory. Logs are large and I/O heavy, so use a separate drive; keep the code directory locked down. Containers write to standard out. Make the location configurable or admins will use symlinks.
- Logging Levels: logs teach humans what normal means. Operations read logs far more than developers do, so log for production operations: ERROR or SEVERE means action is required. Bad user input is at most a warning; a circuit breaker opening or failure to connect to a database is an error. “A NullPointerException isn’t automatically an error.”
- Debug Logs in Production: “rarely a good idea”; the noise buries real issues. Add a build step that removes configs enabling debug or trace.
- Human Factors: logs are a human-computer interface. Three Mile Island operators misread values and took “exactly the wrong action at every turn.” Our failures cost money and reputation, so logs must be clear, accurate, actionable and scannable.
- Voodoo Operations: humans are biased toward detecting patterns even when absent (Shermer: a false negative about a leopard cost more than a false positive), which breeds superstition, and a fix that seems to work becomes lore. His example: a pager message, “Data channel lifetime limit reached. Reset required.”, made an admin fail over the database. It was a debug message he wrote and forgot to disable, about an encrypted vendor channel the application reset itself, about once a week. Six months earlier it was the last thing logged before a database crash: “There was no causal connection, but there was a temporal connection.” With the vague wording it caused weekly peak-hour failovers for six months.
- Final Notes on Logging: include an identifier (user, session or transaction ID) so there is “a string to grep”. Log state transitions even if SNMP traps or JMX notifications also report them: it leaves options open and helps postmortems.
- Instance Metrics: the instance should emit metrics for central collection, perhaps just a periodic stats line in a log file. Many outsource to New Relic or Datadog, so small teams can avoid the substantial effort of running metrics infrastructure.
- Health Checks: metrics take time to interpret, so build in a page or API call showing the app’s own view of its health, “more than just” being up. It reports at least: host IP addresses, runtime version, application version or commit ID, whether it is accepting work, and status of connection pools, caches and circuit breakers. Clients go through a load balancer, not the check itself (see Chapter 9); the balancer uses it to detect crashes and the “go live” transition, when the check goes from failing to passing and the app has finished startup.

Wrapping Up

- Instances are “like cobblestone Minecraft blocks”: not interesting alone, but good ones make a solid large-scale structure. They should be designed for production: deployable, configurable and monitorable.
- Next, connecting instances. The interconnect layer provides many important mechanisms for availability and security, “yet it often gets overlooked.”

> **How this chapter builds the author’s case.** it builds on Chapter 7’s environments and Chapter 5’s stability patterns, uses Chapter 6’s Black Friday case as evidence for transparency, and hands off to Chapter 9 (health checks feed load balancers) and Chapter 10 (system-level transparency).

CHAPTER 9

Interconnect

Place in the argument — why this chapter, why this title. chapter 8 covered instances on machines, but "who is interested in a single instance running by itself?" A standalone process "might as well be on a desert island." The title names the layer of his schematic that knits instances together: routing, load balancing, failover, traffic management. The order runs from choosing a solution by scale, to finding a service (DNS), distributing calls (Load Balancing), refusing excess calls (Demand Control), selecting paths (Network Routing), then dynamic discovery and failover addresses; each step adds dynamism and operational cost, which is why scale comes first.

Chapter opening — untitled section before the first sub-heading in the book

- This chapter continues the "iterative zoom-out": how instances work together, find each other, and are invoked by callers.
- The interconnect layer covers all the mechanisms that knit instances into a cohesive system: traffic management, load balancing, discovery. It is where "we can really create high availability", and as at instance level we need transparency and control: "None of it happens by accident."

Solutions at Different Scales

- Earlier choices depended on the environment; from interconnect up, the right solution depends on the organization, since some techniques need extra software.
- A large team with hundreds of small services does well with Consul or another dynamic discovery service: the cost is amortized over many teams and the rate of change is high enough to justify something dynamic. A small business with a few developers should probably stay with direct DNS entries.
- Why it suits the large company: high rate of change (static consumer configuration would be rewritten several times a day); discovery is another service, raising operational surface area, which a dedicated team can absorb; and no developer knows every other's changes.
- In the small company "the opposite is true in every aspect": slower change, perhaps no operations team.
- Take it with a grain of salt: the balance point moves as big companies' open-source platforms reach small teams. Monitoring once "cost megabucks"; now even the smallest team must have it.
- So judge each solution by the dynamism it supports, how much operational support it requires, and how much global knowledge it requires.

DNS

- For small teams DNS is likely the best choice, particularly with slowly changing infrastructure (dedicated machines, long-lived VMs) whose IP addresses stay stable.
- Service Discovery with DNS: discovery is "more Sherlock Holmes than Siri": find the owners, pry out the names (maybe for a six-pack), put the name in a configuration file. A single name means the provider handles load balancing and availability; several mean the caller balances. Use a logical service name, not a physical hostname, so an alias changes in one place, the name server, not in every consumer.
- Load Balancing with DNS: round-robin, one of the oldest techniques, associates several IP addresses with one service name, so each client connects to one of a pool. It does poorly otherwise: instances must be directly routable from callers; too much control is in the client's hands, and the DNS server, knowing nothing of health, "can keep vending out IP addresses for instances that are toast"; only initial connections are spread, not load; and Java's built-in classes cache the first address, "completely defeating load balancing".
- Global Server Load Balancing with DNS: given those limits it is usually worth moving up the stack, but DNS excels at GSLB, routing clients, usually external, across geographic locations (data centers or

cloud regions) for best performance, bearing in mind that nearby in network terms may not match geography.

- Each location has load-balanced pools, each with an address going to its load balancer; GSLB just gets the request to a pool's virtual IP. Unlike an ordinary DNS server's static database, a GSLB server tracks pool health and offers an address only if the pool passes health checks. Different GSLB servers may return different addresses for the same name, to balance across pools or give the closest point of presence.
- Two levels: globally it rests on DNS and after name resolution is "out of the picture"; the load balancer, a reverse proxy, carries the call. In practice each GSLB server knows all pools but prefers nearby ones, usable for disaster recovery or rolling deployment.
- Availability of DNS: "It doesn't matter how great the service's availability is when callers can't find out how to reach it." Emphasize diversity: not on production infrastructure, more than one provider in different locations, yet another provider for the public status page, and no failure scenario that leaves no working DNS server.
- Remember This: use DNS for services that change rarely; round-robin is a low-cost way to load-balance; discovery is a human process; DNS suits global traffic management with local load balancers; diversity is crucial.

Load Balancing

- Almost everything today is a horizontally scalable farm with request/reply semantics, which helps capacity and resilience but "introduces the need for load balancing": distributing requests across a pool "to serve all requests correctly in the shortest feasible time." This section covers active load balancing, hardware or software inline between caller and providers.
- Balancers listen on virtual IPs (VIPs), dozens per physical port. Each VIP maps to a pool, which defines instance addresses and policy: algorithm, health checks, stickiness if any, and what to do when no member is available.
- It should be transparent: "If the client can tell there's a load balancer involved, it's probably broken."
- Software Load Balancing: the low-cost approach, a reverse proxy. A normal proxy multiplexes outgoing calls into one source IP; a reverse proxy demultiplexes incoming calls and fans them out (Squid, HAProxy, Apache httpd, nginx). Working at the application layer, it is not fully transparent: logged source addresses show only the proxy, so use the X-Forwarded-For header.
- It can also cache responses. Because it sees every request it burdens quickly; if you contemplate a balancer in front of your reverse proxies, look at other options.
- Hardware Load Balancing: specialized devices do the same job, but being closer to the network give better capacity and throughput. They switch at layers 4 through 7, so can balance any connection-oriented protocol and hand off a whole site's traffic to a failover site (with GSLB). The big drawback is price: five digits for low-end, six for high-end.
- Health Checks: the balancer stops sending traffic to an instance failing a configured number of checks (see 'Health Checks' in chapter 8).
- Stickiness: sending repeated requests to the same instance helps stateful services (resources are in memory). Downside: uneven load. Grouping uses a cookie, or the source IP, which breaks badly behind a reverse proxy or a customer's outbound proxy.
- Partitioning Request Types: content-based routing uses something in the URL to pick a pool, such as search to one set and signups to another, or long-running queries apart from fast ones, if the balancer can see the distinguishing feature.
- Remember This: "We cannot treat them as just part of the network infrastructure any more." They affect availability, resilience and scaling, so design for them. If another team manages them as "those things over there", consider your own software layer behind the hardware ones. Software balancers are low cost and easy to operate; hardware reaches higher scale but needs direct network access and

specific skills; good health checks ensure requests can succeed, “not just that the service is listening to a socket”.

Demand Control

- Services now face the whole world and “the world can crush our systems at any time. There’s no natural protection. We have to build it.” Two strategies: refuse work or scale out; here, refusing.
- How Systems Fail: “Every failing system starts with a queue backing up somewhere.” So look at the resources a request consumes and the queues to reach them, to decide where to cut off new requests.
- Sockets: each request uses a socket on every tier, held a little after completion (see ‘TIME_WAIT and the Bogons’). Sockets relate to throughput via Little’s law, depending on duration. Under heavy load services slow, so fewer sockets are free just when most requests arrive: “going nonlinear”.
- NIC bandwidth: Ethernet is serial, so packets queue in finite buffers. Full write buffers block writes; full read buffers stall the connection. The application is slowest to read exactly under high load, another nonlinear effect.
- The listen queue: connections that complete the handshake wait for accept; if they wait too long clients abandon them, and if the queue is full clients retry with delays and give up.
- One edge request may become a tree of service requests, each loading listen queues, sockets and NICs; under load these are held longer, extending response times past the caller’s timeout, and the caller “probably fire[s] a retry at us (exactly when it hurts the worst!)”.
- Preventing Disaster: turn away work we can’t complete in time. This is load shedding, “the most important way to control incoming demand”, and “a quick rejection is better than a slow timeout.”
- Shed as early as possible, so resources are not tied up across tiers; balancers near the edge are ideal. A good first-tier health check tells them when response times exceed the SLA, and they should return HTTP 503 when all instances fail: “too busy, try later.”
- Services can measure their own response time and state (connection pool contention estimates waits) and check dependencies; if a required one is too slow, the health check should show the service unavailable. That provides back pressure through tiers.
- Keep listen queues short. Residence time is queue time plus processing; many measure only processing, so the service thinks all is well while consumers complain. The queue is serial and processing multithreaded, so “queuing time ultimately dominates processing time.” Heuristic start: (maximum wait / mean processing time + 1) x threads, plus 50 percent.
- Because clients retry, a listen queue purge, a tight loop that accepts connections and responds with a canned rejection such as 503 Try Again, can help; it is like a “yellow alert” or “red alert” status.
- TIME_WAIT and the Bogons: a closed socket waits so stray packets time out; otherwise a delayed packet could match a reallocated socket’s sequence number and desync the stream (a bogon). Data-center-only services can set a very low TIME_WAIT.
- Remember This: services must protect themselves. Reject work as close to the edge as possible, since deeper work ties up more resources; provide health checks so balancers protect application code; reject when response time will provoke retries.

Network Routing

- Machines often have several interfaces (a front-end VLAN to web servers, a back-end VLAN to database servers), so the server must know which reaches which destination. Nearby routes follow subnets; distant or third-party services complicate it.
- Operating systems make routing automatic: the primary NIC’s address becomes the default gateway, the first routing-table entry, and the host updates the table from its switches.

- Mostly it “just works,” but trouble comes when multiple routes seem plausible but are not equivalent. Example: PII to a partner over a VPN, where both the VPN and the primary switch may advertise routes.
- Best case you find out in testing, getting destination unreachable: “A consistent error is much better than intermittent success.” Otherwise, depending on advertisement order, packets may go over the public network, and PII goes in cleartext while the service appears to work normally.
- Solutions: static routes (“sometimes they’re the only way”), or software-defined networking, a “network on a network” of virtual IPs, VLAN tagging and virtual switches independent of physical ones, which can assign private IPs, attach DNS names and create firewalls and subnets dynamically.
- Unreliable Enumeration: at one customer, two identical machines numbered their interfaces in different orders, so one’s default gateway was the public-facing switch and the other’s an administrative switch.
- Wrong routing risks reduced availability or exposure of customer data. Record each remote connection’s destination, address and desired route; firewall rules need it anyway.

Discovering Services

- Needed when there are too many services for DNS management, or the environment is highly dynamic; container environments usually hit both.
- Two parts. Announcing: instances announce themselves to receive load, replacing static balancer pools with dynamic ones (any balancer can do this). Lookup: the caller needs at least one IP address, which can look like plain DNS even if a service-aware server supplies it.
- Discovery is another service that can fail or overload, so clients should cache results briefly. Do not roll your own: “there’s a world of difference between writing one that works and writing one that always works.”
- ZooKeeper or etcd can underlie it, wrapped in a library. By the CAP theorem ZooKeeper is a CP system: in a partition (“and there will be a network partition”) some nodes won’t answer or accept writes, so clients need a fallback to other nodes or cached results, which not every client can implement.
- Consul is AP: it prefers availability and risks stale information in a partition.
- The space evolves rapidly; each tool could fill a chapter, and there is no plug-and-play replaceability. The only answer is to “do your homework and commit to solving implementation challenges with whichever tool you choose.”

Migratory Virtual IP Addresses

- Scenario: a critical server fails; the failover node’s cluster server notices the missing heartbeat, starts the application and takes over the virtual IP.
- Virtual IP is overloaded: an address not strictly tied to a MAC address. Cluster servers use it to migrate ownership; load balancers use it to multiplex services onto fewer interfaces.
- The migratory kind moves from one NIC to another, exactly one server claiming it at a time, by associating it with a new MAC address and advertising the route (ARP). It is often used for active/passive database clusters, with clients using the DNS name for the virtual IP, not either node.
- Limit: in-memory state does not migrate, including uncommitted transactions. Some drivers re-execute aborted queries, but updates, inserts or stored procedure calls cannot be repeated, so expect a SQLException at failover.
- In general, a caller must expect the next TCP packet to go to a different interface, causing IOExceptions in strange places, handled differently from plain unreachable errors; if possible retry on the new node (see Circuit Breaker for safety limits on retries).

Wrapping Up

- The interconnect layer is where instances form systems. Key issues: load balancing, routing, load shedding and service discovery.
- Existing solutions may be there to plug into; powerful tools need operational support costly for one team.
- Next: control over the extended mix of instances and tools, to deploy, monitor and intervene in production.

> **How this chapter builds the author's case.** it moves from chapter 8's single instance to the layer joining instances, uses chapter 8's health checks as input to balancers and load shedding, and hands on to the control plane.

CHAPTER 10

Control Plane

> **Place in the argument — why this chapter, why this title.** The title names the layer above the instances. Earlier chapters worked "up from bare metal through layers of abstraction and virtualization to create a sea of instances running on machines"; the control plane is what must "put these pieces in the right place and knit them together". The chapter follows the stack because it needs those layers first, and it hands on to security as "one last area". Section order: judgment (how much to build), a cautionary outage, roles, seeing (transparency), configuring and deploying, intervening (Command and Control), then the all-in platforms. "Remember This" gives the same order: visibility first, then leverage, then control.

Chapter opening — untitled section before the first sub-heading in the book

- The pieces are "scattered around like an upended box of LEGO blocks".
- Definition: the control plane is all the software and services running in the background to make production load successful. Test: "if production user data passes through it, it's production software. If its main job is to manage other software, it's the control plane."
- The solution space is "not well partitioned among tools, packages, and vendors"; no single package does everything, so expect integration effort and trial and error.

How Much Is Right for You?

- Every part is optional if you accept trade-offs. Without logging and monitoring, postmortems, recovery and defect discovery take longer or are not done; if you can live with finding out from a call from the CEO, skip it. A static physical network needs no IP management; with overlay networks and VLANs "you'll go mad without IP management".
- More sophistication costs more; every piece is ongoing operational cost. Small and slow-changing: probably not worth it. A platform team amortized across hundreds of services deployed hundreds of times a year: makes a lot more sense.
- The equation moves: logging and monitoring was almost entirely commercial in 2007, now almost entirely open source; today the hardest problem is choosing among alternatives.
- Bottom line: don't install one of everything you read about, but keep re-evaluating overhead and difficulty.

Mechanical Advantage

- **Mechanical advantage** is the multiplier on human effort that simple machines give. It "works for good or for ill": releasing to ten thousand machines is good, but automation goes wrong.
- Example: on February 28, 2017, AWS S3 in US-East-1 went down for about two hours; tens of thousands of companies with hard dependencies on S3 had outages for many more hours.
- A postmortem review has three jobs: explain what happened, apologize, commit to improvement. Amazon's does all three.
- System Failure, Not Human Error: an authorized team member using an established playbook ran a command meant to remove a few servers; one input was wrong and a larger set was removed. Nygard sympathizes but notes human error appears nowhere. "This is not a case of humans failing the system. It's a case of the system failing humans." Tools and playbooks amplified a minor error; "system" means S3 plus the control plane software plus human processes.
- The playbook had apparently worked before. Learn from successes: who ran it, were error-checking steps relaxed over time, what feedback did the system give?
- Anomalies labeled root cause after an outage also exist in ordinary operations; hindsight gives them weight. So hold postmortems for successful changes and find the near misses and what safety net could have helped.

- Automation Goes Really Fast: Amazon says the tool "allowed too much capacity to be removed too quickly" and now removes capacity more slowly with minimum-capacity safeguards. Reddit's August 2016 outage was similar: its autoscaler read a partially migrated ZooKeeper database claiming few servers were needed and dutifully shut down the rest.
- Common thread: automation is not simply enacting an administrator's will but is "more like industrial robotics": it senses current state, compares it to desired state, and effects changes to close the gap.
- One machine of thousands is no big deal, but enough shutdowns dent capacity. Past 50 percent of capacity the automation ought to pause for human confirmation.
- "Automation has no judgment." By the time a human perceives the problem, "it's a question of recovery rather than intervention". Use automation for what humans are bad at (repetitive tasks, fast response) and humans for what automation is bad at (perceiving the whole situation).

Platform and Ecosystem

- Monitoring: "The monitoring team doesn't do the monitoring, it provides the ability for others to do their own monitoring": a shift from ownership of the domain to offering a service to customers.
- Old way: the team sits in the change loop, with a "request for monitoring" form. New way: work "one level removed", building tools where developers give a JSON description of their monitors. Like an OO interface: implementation can change as long as the contract holds.
- DBAs: administrator and architect were blurred, so the data model was contorted to suit the server; much NoSQL energy was about refactoring that. With NoSQL the platform team keeps the database healthy and "the data model is up to the application".
- With SQL RDBMSs it is too easy for one application's schema change to harm others, so decree a separate physical database per service: not resource-efficient, but teams move independently. A shared database can work if developers and DBAs agree on a simpler machine-readable migration format (XML, JSON, YAML), since parsing SQL for sanity checks is too hard.
- The platform team's goal is to enable customers and take themselves out of the loop on day-to-day processes. If technology or architecture makes that hard, "it's a good argument to change your technology!"

Development Is Production

- The typical dev server is a ramshackle mess; QA has integrations stubbed out and unrealistic data (production has PII). Can you trust passing QA? If dev is a fresh known VM and QA is stamped out by production's automation with recent anonymized data, "you're doing quite well".
- Most treat development environments "like a shantytown". There is no right number of QA environments; virtualize them so every team can create its own on demand.
- Odd, because developers' work runs on services (version control, CI, QA, repository); if the copywriters' CMS went down, that would be at least Severity 2.
- So these tools and environments need production-level SLAs: "The development platform is the production environment for the job of creating software."

System-Wide Transparency

- This extends 'Transparency' (instance state) to a system-wide picture. Two fundamental questions: are users receiving a good experience, and is the system creating the economic value we want? "Is everything running?" is not on the list; at scale "partially broken" is the normal state.
- Real-User Monitoring: instance metrics cannot tell you user experience without a model of circuit breakers, caches and fallbacks, so measure it directly: **real-user monitoring** (RUM), with apps reporting timing and failures to a central service.
- Hosted services (New Relic, Datadog) win on rapid startup, wide agent coverage and polished dashboards; the downsides are fees that scale with you, costly switching, and companies unwilling to

send monitoring data over the Internet. On-premise tools lose rapid startup. Open source avoids the visible fee but has integration challenges, less polish and hidden labor costs.

- It matters little which you choose: “focus on adopting your chosen solution thoroughly. Don’t leave any ‘dead zones’ in your system.”
- Economic Value: transparency is where we most directly perceive the link between systems and financial success. Value is harmed by bad user experience and by high cost: the top line and bottom line.
- Top line: watch each step of a business process for drop-off or exceptions, and watch queue depth, “your first indicator of performance degradation”; queuing time directly hits revenue.
- Bottom line: costs come from elastic infrastructure (unchecked autoscaling, runaway automation), operations (incident response is unplanned work), and runtimes, but optimize only services that matter.
- Look ahead too: bottlenecks that will block growth, overscaled services. Seeing monitoring as about economic value makes it easy to decide what to monitor.
- The Risk of Fragmentation: marketing, sales, operations and development use different tools (development uses “blind hope and intuition”). Different constituencies need different views but “the same information system overall”, as “How’s the weather?” means different things to a gardener, a pilot and a meteorologist.
- Logs and Stats: collectors gather instance data. Push mode (syslog) suits containers; pull mode has a central collector remote-copy local logs. The real beauty is indexing: search, trendlines, alerts.
- Metrics are similar but may need a sampling program on the target; they can be aggregated over time (seconds today, hourly after a week), saving disk and making queries over large spans possible.
- What to Expose: you will likely guess wrong about which metrics matter, and “the bottleneck that burns you next year probably doesn’t exist right now”, yet exposing everything is unbounded. Useful categories: traffic indicators; business transactions; users; resource pool, database connection, integration point and cache health; data consumption. All counters imply “in the last n minutes” or “since the last reset”.
- Each metric has a range: nominal within it, with a caution range near a threshold. A rule of thumb for continuous metrics: the mean for the period plus or minus two standard deviations. The most stable period is usually the “hour of the week” (2 p.m. on Tuesday); in travel, floral and sports, count back from an event. “There is no one right answer for all organizations.”

Configuration Services

- ZooKeeper and etcd are distributed databases for coordinating configuration, which includes “the arrangement of instances among themselves”: orchestration, leader election, quorum-based consensus.
- They are “built with code and not magic”, bound by the CAP theorem. Scalable but not elastic: adding nodes degrades response while data rebalances.
- Clients will sometimes be unable to reach the service, or its nodes unable to reach each other; running with slightly outdated configuration must be safe, or you must shut applications down during a partition.
- Instances can report version numbers or commit SHAs so a script can reconcile actual with expected state after deployment. But every write goes through consensus; fine for slowly changing data, and “they definitely don’t stand in for a log collection system”.
- Pointers: instances must start without the service and keep working when it is unreachable; a partitioned node must not shut down the world; replicate across regions.

Provisioning and Deployment Services

- Part III covers designing for deployability; here is the supporting infrastructure. Deployment is the nexus of development and production and often “ridiculously painful”, so a good place to start.
- Push tools use SSH or an agent to run scripts on machines that may not know their roles; pull tools rely on machines that know their role and fetch the latest bits from a configuration service. Pull suits elastic scaling (ephemeral identities); push can be simpler for long-lived hosts, using commodity SSH.
- Add a package repository for blessed binaries “that isn’t populated from a developer’s laptop”. Build on a clean server with libraries of known provenance. Repeatable builds make what works on your machine work in production.
- Build Server as Attack Vector: build servers (Jenkins, Bamboo, GoCD) get attacked. A vendor was hit via a compromised CI plugin that injected code targeting a well-known customer; it controlled libraries but overlooked build-system plugins.
- Canary deployments: a small set of instances gets the new build first and coexists with the old (see ‘Handling Versions’); if they behave oddly or metrics go south, the build is not rolled out. The purpose is “to reject a bad build before it reaches the users”.
- At larger scale a placement service decides how many instances to run, spreads them across regions, and drives IP addresses, VLANs, load balancers and firewall rules. Then look at the platform players; your software must include a description of its needs and wants for the platform.

Command and Control

- Live control is only necessary if instances take long to be ready. If a configuration change took ten milliseconds and a restart a hundred, it would be simpler to kill the instance and let the scheduler start a new one. Containers with a configuration service are that world.
- Many are not: the JVM needs JIT warm-up, caches must fill, and VMs take minutes to restart. Then send control signals to running instances.
- Controls to Offer (a checklist, not all needed): reset circuit breakers; adjust connection pool sizes and timeouts; disable specific outbound integrations; reload configuration; start or stop accepting load; feature toggles.
- Schema-update or delete-and-reseed controls are hazardous in production and come from a breakdown in roles (developers don’t trust operations; operations bars developers). “Don’t build a self-destruct button into your production code!” A “flush cache” button “vents all your atmosphere into space”.
- Sending Commands: the simplest is an admin API over HTTP on a different port from ordinary traffic, never public. cURL is fine at first.
- At scale, scripts stall: at a quarter-second per call, 500 instances take two minutes, and a non-responding instance stalls the loop. Build a “command queue”, a shared queue or pub/sub bus all instances listen to. It makes dogpiles easier, so add random delay and target waves or gangs.
- Scriptable Interfaces: “Admin GUIs demo very well. Unfortunately, they are a nightmare in production.” Clicking can’t be scripted and repeats per instance; a clean shutdown needing minutes of clicking on each of six servers, within a one-hour change window, was rarely observed. The best long-term interface is the command line, where operators build scripts, logging and automated actions.
- Remember This: “Anything you build must either be maintained or torn down”, so match team size and workload scale. Start with visibility (logs, tracing, metrics; index logs and get them off machines for postmortems). Then use configuration, provisioning and deployment services for leverage over larger or more dynamic systems; this pipeline “is the production environment that developers use to produce value”. Once the system is somewhat stabilized and problems visible, build control mechanisms; a large system on long-lived machines benefits more than a highly dynamic one.

The Platform Players

- The earlier solutions need "some assembly required": you can adopt incrementally, but optionality costs plumbing time, for example getting authentication and role-based authorization working together and integrating monitoring into a unified view.
- At the other end: "The platform is to the data center what the operating system is to the personal computer." It abstracts infrastructure, schedules tasks across computers, and assures its parts work together coherently.
- Contenders at writing: Kubernetes, Mesos, CloudFoundry, Docker's Swarm Mode; the population persistently permutes. Unlike cloud providers, platforms install anywhere (on premises, hosting facility, public cloud).
- Platforms need care and feeding, so a big group adopts one; individual teams lack capacity or authority to build their own.
- If adopting one, embrace it: no arm's-length use, no wrapping the API or your own scripts.

The Shopping List

- Not every organization needs everything; apply cost/benefit to each: log collection and search; metrics collection and visualization; deployment; configuration service; instance placement; instance and system visualization; scheduling; IP, network and firewall management; autoscaler; alerting.

Wrapping Up

- "Every solution creates new problems." We virtualized everything, so workload spans containers, VMs, clouds and data centers, and keeping tabs on it needs new tools.
- Visibility answers the two questions (good user experience; economic value), using information collected across instances and tracing tools to find bottlenecks and points of failure.
- Then intervene: control systems and configuration services change running instances; scheduling and deployment tools change the instance assortment.
- "Automation makes everything go faster" yet lacks judgment, so failures come quickly: "We need to build safety mechanisms into the automation itself." Last area: security.

> **How this chapter builds the author's case.** It builds on the "sea of instances" of the preceding chapters and on 'Transparency' (instance state), scaling it to the system. It reuses earlier warnings about automation ('Force Multiplier'). It points forward to Part III (deployment, 'Handling Versions') and ends by handing over to security.

CHAPTER 11

Security

> **Place in the argument — why this chapter, why this title.** Security is the last area in the layers of design for production (chapter 10 ends by pointing here) and closes Part II, the "slow zoom out from the physical substrate" to "systemic considerations". The opening insists "Security must be baked in. It's not a seasoning to sprinkle onto your system at the end." The scope is narrowed to topics that "earned their place by sitting in the intersection of software architecture, operations, and security". Order: the OWASP Top 10 (real attacks), least privilege (what a process may do), configured passwords (secrets the system must hold), then security as an ongoing process. Deployment comes next.

Chapter opening — untitled section before the first sub-heading in the book

- Poor security can damage your organization and many others: fraud, extortion, remediation, customer compensation, regulatory fines, lost reputation, and jobs up to the CEO's.
- In 2017 WannaCry hit more than 70 countries; at the UK's NHS it canceled X-ray sessions, closed stroke centers and postponed surgeries, which "put lives at risk". Equifax revealed 145.5 million US identities stolen; Yahoo! announced 3 billion accounts.
- Breaches can implant data (false identities, shipping documents), as perhaps in California's 2013 nut theft crisis.
- A dedicated security team doesn't take you off the hook: "You're still responsible to protect your customers and your company."
- Plan: OWASP's top ten, plus data protection and integrity.

The OWASP Top 10

- OWASP has catalogued incidents since 2001 from member organizations' real attacks, so real lessons. The list is a consensus on the most critical web application flaws, updated every three or four years; this one is based on Release Candidate 1 and may not be adopted: "you can't ever stop worrying about security".
- Injection: an attack on a parser or interpreter that relies on user-supplied input. Classic: SQL injection ("Little Bobby Tables"), where concatenated strings let input end the intended query and add DROP TABLE, with a double hyphen commenting out the rest.
- "There's no excuse for SQL injections in this day and age": every SQL library allows placeholders, so use a prepared statement with ? and setString rather than concatenation. Any service that bashes strings together into queries from user input is vulnerable, and "comes from a user" includes database data that originated from a user.
- XML external entity (XXE) injection: documents can define external entities in the DTD that act like include statements, so the parser inserts whatever the URL returns. The attacker hopes the error response echoes the expanded input. Most parsers are vulnerable by default and must be configured; the answer is not to parse XML yourself with regular expressions.
- SQL and XXE are two of many ways; injection has held the top spot since 2010.
- Broken Authentication and Session Management: from session IDs in URLs to unsalted passwords (if stored unhashed, "please stop reading now and go fix that").
- URL session IDs are visible to every switch, router and proxy; copy-pasting a link shares the session. An electronics retailer's special-offer email carried the marketer's session ID; thousands of users tried that session and the front-end servers each tried to take exclusive ownership, causing an outage.
- Session hijacking: a plain-text ID can be sniffed. If unlucky, the victim is an administrator in a web GUI. It can occur with cookies too, via XSS.

- Session fixation: the attacker gets a valid ID, sends the victim a link containing it, and after the victim authenticates both have full access. Session prediction: IDs from a predictable process can be guessed; sequential IDs are “the absolute worst choice”, and looking random isn’t being random.
- OWASP guidelines: long IDs with lots of entropy; a cryptographic PRNG (your language’s rand() probably isn’t); protect against XSS; fresh ID on authentication; use the patched platform session management; exchange IDs only in cookies and disable query parameters.
- Credentials sent in the clear is still the most common problem, from two toxic practices: TLS certificates are hard and easy to install wrongly, and tools leave the trust store to the user, so we write services over HTTP.
- Authentication verifies the caller’s identity. The old pie crust defense (authenticate at the boundary, then services call freely) no longer fits, so authenticate everywhere; don’t trust IP addresses.
- Do’s and don’ts: don’t keep passwords in your database; never email one; use a strong hash with salt; allow long passwords and pasting; plan to rehash and change the salt; don’t allow unlimited attempts. But the instinctive limit of three before lockout is unfriendly because devices and apps retry automatically with old passwords.
- First-party authentication: we keep the credentials database and check what the principal provides. Third-party: the principal presents proof from another authority, which we verify and check was not intercepted;
- Cross-Site Scripting (XSS): a service renders user input into HTML without escaping. Like injection, it exploits our representation of structured data as ordinary characters, via premature delimiters and unwanted commands. Example: an echoed search parameter closes the input tag and adds a script that sends document.cookie to the attacker.
- Front-end apps that put service content straight into the DOM are equally vulnerable. Hostile JavaScript in a contact-us form can run in an administrator’s browser weeks later; log viewers can be hit too.
- Scanners find XSS quickly: “Expect an exploit within milliseconds.” XSS can conscript your system to attack others.
- Bottom line: “never trust input. Scrub it on the way in and escape it on the way out.” Secondary lesson: don’t build structured data by smashing strings together; use a library that escapes everything and forces you to ask nicely to do unsafe things.
- Broken Access Control: attackers reach data they shouldn’t. Direct object access: a sequential database ID in a URL lets an attacker probe other customers’ data. Two-part solution: deter URL probing and authorize access to objects.
- Deter URL Probing: no database IDs in URLs; use non-sequential unique identifiers, or a session-sensitive URL (users/me), or a session-specific map from random IDs to real ones (unknown IDs give 404). Downside of the map: links don’t persist across sessions, violating REST.
- Authorize Access to Objects: services confuse “possesses a URL” with “allowed to access resource”. Check on every request: “URLs are just text strings, and anybody can create whatever URL they like!”
- Leak: 404 for nonexistent but 403 for unauthorized-but-existing tells attackers what exists (customer counts by ID, or whether an email address is a customer). Rule: “If a caller is not authorized to see the contents of a resource, it should be as if the resource doesn’t even exist.”
- Directory traversal: input with ../ concatenated into a file name opens files outside the base directory; with uploads, it overwrites any file the service can modify (another reason not to run as root). Safe handling: treat the client’s filename as an arbitrary string in a database field and use a generated random key as the real filename. See CWE 22.
- Security Misconfiguration: default passwords (admin/admin) and, usually by omission, unneeded default features left as unmonitored entry points. Seek out admin consoles and never allow default passwords in production; watch container images that bundle Redis, Mongo, Postgres or ZooKeeper.

- Early 2017: over 20,000 MongoDB installations with default credentials exposed to the Internet were wiped and held for bitcoin. “Remember the install script is the first step in installation, not the last.”
- Split internal traffic onto its own NIC (see ‘Programming for Multiple Networks’) to reduce the attack surface, meaning all IP addresses, ports and protocols reachable to attackers. Use personal, not group, administrator accounts, and log administrative calls: “you’ll be one of the few people to witness a smiling auditor.”
- Sensitive Data Exposure: the big one (credit cards, medical records, emails). Hackers “look for cracks in your shell” rather than breaking crypto: a stolen laptop with a spreadsheet extract, or TLS at the edge and plain HTTP inside, another pie crust.
- Guidelines: don’t store what you don’t need (use a card tokenizer); HTTP Strict Transport Security; stop using SHA-1; never plain-text passwords; encrypt sensitive data in the database (“a pain, but necessary”); decrypt based on the user’s authorization, not the server’s.
- AWS KMS manages master keys and data encryption keys encrypted by a key encryption key, so decryption keys aren’t left lying around; on premises, HashiCorp’s Vault.
- Insufficient Attack Protection: a firewall is not enough: “We must always assume that attackers have unlimited access to other machines behind the firewall.” Services don’t track bad requests by origin or block heavy offenders, so attackers can keep probing. Log bad requests by source principal and collate them (see ‘Logs and Stats’).
- A whitelist of consumers isn’t feasible since consumers deploy independently, but a blacklist is. API gateways block by API key and throttle, slowing data compromise in an attack; layer 7 firewalls carry attack signatures.
- Cross-Site Request Forgery (CSRF): less of a problem now since frameworks defend, but old applications remain, as targets or stooges. A page on another site makes the user’s browser access yours with the usual cookies, so “just because the user appears to have a logged-in session doesn’t mean the request is intentional.”
- Defenses: don’t let your site launch CSRF (XSS is a trap); anti-CSRF tokens on requests with side effects (frameworks do this but may need enabling); the SameSite cookie attribute, sent only when origin matches target, though browser support was incomplete in June 2017 and it may force changes to session management. The RFC suggests a not-same-site session read cookie for GET and a strict same-site write cookie for state changes.
- Using Components with Known Vulnerabilities: a Struts 2 remote code execution flaw is what got Equifax. “Most developers don’t even know what all is in their dependency tree.”
- Most attacks are mundane: tools probe IP addresses for hundreds of vulnerabilities, some ancient.
- Extract a report of all build artifacts (including the build tool’s plugins), check it weekly against CVEs, or better break the build on a CVE with a plugin, or use VersionEye. Many vulnerabilities never get CVEs, so watch mailing lists and issue trackers.
- Underprotected APIs: the newcomer. APIs became a primary architectural concern, and for some companies are the whole product. Scanners are slow here because there’s no standard metadata: “if you can’t tell how it should work, how do you know when it’s broken?” And attack tools with the right credentials look like legitimate users.
- Defense one is bulkheading (see ‘Bulkheads’): stolen credentials from one customer must not reach other customers’ data. An API cannot use hyperlinks as a security measure; a client may issue 10,000 requests to learn your URL template, so authorize the link going out and reauthorize the request coming in.
- Two: the most secure communication (TLS rejecting protocol downgrades, up-to-date root CA files, bidirectional certificates between businesses). Three: harden the data parser, using a generative testing library to feed it bogus input; fuzz-testing matters because APIs respond as quickly as possible to as many requests as possible, making them “savory targets for automated crackers”.

The Principle of Least Privilege

- A process should have the lowest level of privilege needed, never root or administrator. Example: Windows servers left logged in as administrator for weeks because ancient vendor software required it.
- Root software is automatically a target; once root is cracked, the only safe course is to reformat and reinstall. Give each major application its own user. The only UNIX need for root is opening a port below 1024, and a web server behind a load balancer (see 'Load Balancing') can use any port.
- Containers and Least Privilege: a container per application keeps filesystems apart. But images often include most of an OS or a whole init system, a large attack surface, and patch tools don't handle containers, so they keep vulnerabilities IT patched weeks ago.
- Solution: treat images as "perishable goods", rebuilt automatically from an upstream base and your code, ideally in the CI pipeline, with timed builds for applications no longer in active development.

Configured Passwords

- Passwords are "the Brazil nut of application security; every mix has them, but nobody wants to deal with them." Nobody can key them in at every startup, so credentials live in persistent files and are vulnerable. A customer-database password is worth thousands to an attacker, so protect it at the highest level achievable.
- At minimum: keep production database passwords apart from other configuration and out of the installation directory (operations sometimes zip it up and send it to development); readable only by the application user (or root-owned if the application drops privileges after reading).
- Vaulting reduces the problem to securing one key, but is not a complete solution; use intrusion detection such as Tripwire on permissions. With KMS, applications fetch decryption keys by API so encrypted passwords and keys aren't stored together; Vault holds credentials directly.
- Expunge keys from memory quickly, since dumps contain them: disable UNIX core dumps in production; a Windows blue screen dump can hold the entire physical memory, "passwords and all".

Security as an Ongoing Process

- "Frameworks can't protect you from the Top 10. Neither can a one-time review by your company's AppSec team." Encrypted communication, encryption at rest, authentication and authorization are cross-cutting concerns in the architecture.
- New attacks emerge constantly: have a process to discover them (hopefully before they are used on you) and remediate quickly. New technology will have vulnerabilities; don't give up its advantages, but patch vigilantly and be able to redeploy servers on a moment's notice.

Wrapping Up

- Application security affects life and livelihood; consider components and the whole system: "Two secure components don't necessarily mix to make a secure system."
- User data, especially credit cards, is the most common target, but industrial espionage is real and can look as harmless as the location of a shipment of tasty pecans.
- "Beware the pie crust defense." Protect internal APIs with authentication and authorization and encrypt data on the wire even inside an organization. "There's no such thing as a secure perimeter today"; breaches can be present a long time before detection.
- Full treatment is beyond the book; follow the trail into CVEs, CWEs and CERTs. This finishes the slow zoom out from copper, silicon and iron oxide to systemic considerations; next is deployment, "the moment of truth".

> **How this chapter builds the author's case.** It adds the adversary to the production system built up in earlier chapters, reaching back to 'Programming for Multiple Networks', 'Bulkheads', 'Logs and Stats' and 'Load Balancing', and to the previous chapter's build-server and automation warnings. The pie

crust warning and “Two secure components don’t necessarily mix to make a secure system” tie it to the book’s systemic view, and it hands over to deployment.

PART III OVERVIEW

Part III — Deliver Your System

Author's description (Preface): "Pouring bits onto servers" is easy; the hard part is frequent small changes without breaking consumers. The Part covers designing for deployment, deployment without downtime, and versioning across services.

The theme of the Part

- Deployment is something to design for, and changes must not break the systems that talk to yours.

How the chapters build the theme

- Chapter 12, Case Study: Waiting for Godot shows the deployment army as an antipattern.
- Chapter 13, Design for Deployment: automated and then continuous deployment.
- Chapter 14, Handling Versions: change at the boundary, providing and consuming versions.

What the Part contributes to the book

- Easy, routine change; Part IV asks whether the system can adapt over its whole life.

CHAPTER 12

Case Study: Waiting for Godot

Place in the argument — why this chapter, why this title. A case study opens Part III (Deliver Your System). Its first line sets the theme: “Nothing is done until it runs in production.” The title signals waiting with nothing happening, and the author closes by naming what he showed as an antipattern and promising the following chapters will explain its forces and the way out.

Chapter opening — untitled section before the first sub-heading in the book

- Sometimes the path to production is a smooth highway; with older systems it can be a muddy track of potholes, bandits and checkpoints. “This was one of the bad ones.”
- Narrated as a night deployment. At 1:17 a.m. the wall clock means little; the playbook’s Lamport clock is stuck near 11:50 p.m. because the SQL scripts are not finished. Playbook time and solar time must converge before dawn.
- The playbook began the afternoon before with status reports from every area and a 3 p.m. go/no-go meeting. Everyone said go, though QA had not finished testing. Stakeholders were then told to sleep so they could smoke test at 1 to 3 a.m. (the UAT window).
- The playbook is itself reviewed row by row two days ahead, two or three hours per meeting. Over 40 people are consumed across 24 hours, more than twenty in the room and as many on a conference bridge.
- The author is Sys Ops: his script, run from the jumphost, updates a symbolic link to the new code, runs the JSP precompiler and starts the servers. Then he waits. The one bug report concerned a page not in the release.
- Napkin math: about $100 per person-hour times the army for a day is about $100,000 per deployment, done four to six times a year.
- Contrast: years later an investor at Etsy pushed the button to run its “deployinator”. The author felt a “deep sense of loss”: people used as if they were bots.
- Outcome: the deployment failed UAT. QA data did not match production, which had extra content with JavaScript that broke on the new page structure. Rollback finished around 5 a.m.; the second attempt was planned for two days later.
- Conclusion: “You may have a deployment army of your own”, more likely the older the software. Faster, more routine deployments bring an immediate financial benefit and a virtuous cycle; people can stop doing “jobs that should be scripts”.

How this chapter builds the author’s case. It supplies the antipattern the next chapters take apart: the forces behind it, then the climb out.

CHAPTER 13

Design for Deployment

> **Place in the argument — why this chapter, why this title.** The title says deployment is something to design for, not just something operations does afterward. The chapter follows directly from the Godot case study ("In the last chapter, we were stuck in a living nightmare") and turns to "sweeter dreams": automated, then continuous deployments. It promises packaging, integration point versioning and database schemata. The sub-headings run from vocabulary and premise, to the tools, to the practice, to the detailed timeline (Phases of Deployment), then the moral.

Chapter opening — untitled section before the first sub-heading in the book

- The previous chapter's endless deployments waste "countless hours and dollars". This chapter teaches how to design applications for easy rollout, covering packaging, integration point versioning and database schemata.

So Many Machines

- Terms: **machine** is a stand-in for a "configurable operating system instance" (physical host, virtual machine, container or unikernel). **Service** is a callable interface for others to use, always made of redundant copies of software on multiple machines.
- There are more ways to run software than ever, so environments have more machines, mostly virtual. Some are short-lived "mayflies", some created by other machines, so more configurations and tools to manage.
- His point: if we accept this complexity, "we should certainly get something back out of it in the form of increased uptime during deployments."

The Fallacy of Planned Downtime

- Premise of the book: version 1.0 is the beginning of the system's life, so plan for "many upon many" deployments, not one or a few. Old way: throw code over the wall; operations schedules "planned downtime".
- "I hate the phrase 'planned downtime.'" Nobody tells users the plan; "to the users, downtime is downtime". Releases should be like Agent K in Men in Black: users get on with their lives because they do not know about it.
- The design error: we design for the state after a release, as if the whole system changed in one "instantaneous quantum jump". Updating takes time, and users see the system in the during state, not just before or after.
- The answer: design applications to account for the act of deployment and the time the release takes. "We treat deployment as a feature." Three concerns follow: automation, orchestration and zero-downtime deployment.

Automated Deployments

- The tools are described only to give a baseline for the design forces they impose.
- Build pipeline: it starts after a commit to version control, like a CI server (often implemented with one), with unit tests, static analysis and compilation. Where CI stops at a test report and an archive, the pipeline continues to deploy to a trial environment, run migration scripts, run integration tests and culminate in a production deployment.
- It is "more like a build funnel": each stage looks for reasons to reject the build.
- Tools: "dodge the analysis trap" and pick one that suffices.
- Configuration management tools (from Chapter 8) share attributes. You declare a desired configuration in text files that can be version-controlled. They describe a desired end state, not the actions as a shell script would, and the tool works out the actions.

- Mapping configurations to machines is either manual (the operator assigns each host a role) or automatic (the operator asks for "Service X should be running with Y replicas", which suits a PaaS). The platform then finds hosts with enough CPU, RAM and disk, avoids co-locating instances, and configures load balancing and routing.
- Two packaging strategies. Convergence installs onto a minimal booted image with reusable, parameterizable scripts, and the tool examines current state and plans a way to the declared one, with ordered dependencies. Immutable infrastructure makes a fully built VM or container image in the pipeline; any extra configuration is injected by the environment at startup (AMI "user data").
- The immutable camp argues that "convergence never works": a long-lived machine may be in a state the tool cannot repair, and parts of its state (kernel parameters, TCP timeouts) are not in the recipes. Always starting from a known master image "should succeed every time" and gives recipes one initial state to test. To change, update the scripts, build a new machine and delete the old one.
- Immutable infrastructure aligns with IaaS, PaaS and automatic mapping; convergence with physical deployments, long-lived VMs and manual mapping. "Immutable infrastructure is for cattle, convergence is for pets."

Continuous Deployment

- Between commit and production, "code is a pure liability": undeployed code is unfinished inventory with unknown bugs, possible scaling or downtime problems, or a feature nobody wants. Continuous deployment reduces that delay to minimize the liability.
- Vicious cycle: a longer delay means more changes per deployment, hence more risk; the natural reaction is more review steps, which lengthen the delay and raise the risk further.
- The only way out: "If it hurts, do it more often." In the limit that means running the full build pipeline on every commit.
- Variation at the final stage: automatic production deployment, or a "pause" stage where a human affirms the build is good. If the cost of moving slower exceeds the cost of a deployment error, lean automatic; in a safety-critical or regulated setting, lean toward a human check. Either way an authorized button-pusher must be available at any time, even for a 2 a.m. emergency change.

Phases of Deployment

- Spectrum of grain: PHP (copy files, worry only about partial copies), a five-million-line Java EAR with caches and pools to initialize, whole VM images. The larger the grain, the longer to apply and activate, which a rolling deployment must account for. "It's no good to plan a rolling deployment over a 30-minute window only to discover that every machine needs 60 minutes to restart!"
- Two time scales nest. The macroscopic scale is the whole rollout (preparation, rollout, cleanup). The microscopic scale is a single instance, with four time spans: prepare, drain, apply, and startup until ready.
- Prepare: copy files so a symlink can switch (mutable), or deploy the new image (immutable). Drain: seconds for a stateless microservice, but for sticky sessions session timeout plus maximum session duration, possibly unbounded; blocked threads block the drain. Watch the load or use a "good enough" time limit, more so at larger scale. Apply: quick for a symlink, error-prone if manual; for disposable infrastructure it means a new instance. Ready: more than runtime startup, since caches, JIT and connections must be warm, or early requests fail or crawl.
- Once deployment is seen as a span of time, "we can enlist the application to help with its own deployment", smoothing over what normally causes downtime: schema changes and protocol versions.
- Preparation is everything done without disturbing the old version, still running everywhere. It is safe to push new content and assets if they have new paths or URLs.

- Relational Database Schemata: schema changes drive “planned downtime”. Use a migrations framework for programmatic control, but it does not make changes forward- and backward-compatible; you must split them into expansion and cleanup phases.
- Expansion is safe before the code rolls out because the current application does not use any of it: add tables, views, nullable columns, aliases or synonyms, stored procedures, triggers, and copy existing data.
- **Shims**: “In deployments, a shim is a bit of code that helps join the old and new versions of the application.” Example, a split table: during rollout some instances write the old table and some the new, so an entity written to the old table just before shutdown would be lost. Insert, update and delete triggers in both directions bridge them, avoiding infinite loops. Half a dozen shims per change is “the price of batching up changes into a release”; smaller releases cost less (see trickle-then-batch below).
- Test shims on copies of real production data, not “nice, polite, QA-friendly data”. Old records can hold NULLs in now-required fields; never rely on what the application currently says is legal.
- Schemaless Databases: “A schemaless database is only schemaless as far as the database engine cares.” Old documents, back to the first customer record, may be unreadable by the evolved application, “time bombs” that raise an exception on load.
- Three approaches. First, write the application to read every version ever created, through a translation pipeline with one translator per version, a reader that detects the version and dispatches, and splits where a document was split. It needs tests for all permutations, with old documents kept as seed data, and translation time grows with depth.
- Second, a migration across the whole database during deployment. This works while data is small, but later takes hours, so the application must read both versions. With rollout and migration concurrent, the only bad case is an old instance reading a new document, so roll out the application before running the migration.
- Third, the one he likes best: “trickle, then batch”. The new version migrates documents as they are touched, amortizing the migration over requests at a bit of latency per request. After a while, batch-migrate the untouched remainder, safe concurrently because no old instances remain; then deploy again to remove the conditional check. Restriction: do not run two overlapping trickle migrations on the same document type, which may mean splitting a big design change across releases. It applies to any big migration too long for a deployment.
- Web Assets: front-end asset versions are tightly coupled to back-end changes, so users must receive assets compatible with the instance they use.
- Static assets get far-future cache expiration (ten years). On deployment, cache busting makes browsers and proxies fetch the new version. He prefers a git commit SHA in the URL path rather than a query string, so old and new sit in different directories. Stripping the version with rewrite rules “assumes a big bang deployment and an instantaneous switchover”.
- If assets and application come from the same server, a browser may get a new page from an updated instance but be balanced onto an old instance lacking the new assets. Two fixes: session affinity, or deploy all assets to every host before activating new code (which gives up the immutable style). “In general, it’s probably easier to just serve your static assets from a different cluster.”
- Rollout, convergence style: update in batches, since all-at-once cannot keep up with demand and one-at-a-time is too slow. With groups Alpha to Foxtrot: stop new requests to a group, wait for drain, update, wait for green health checks, resume requests, repeat.
- The first group is the “canary” group: ramp traffic gradually while watching errors, latency and RAM, and shut traffic off to investigate anomalies.
- Rather than abruptly removing a machine from the load balancer, use an end-to-end “health check” route on every application and service. A good one reports versions, host IP and the status of pools, caches and circuit breakers. An “available” flag tells the load balancer to send no new work while

existing requests finish, and the service should start with it false because listening on a socket is not being ready.

- While Charlie is updated, Alpha and Bravo are done and Delta and Foxtrot wait: “Both the old and new versions are running at the same time”, which is where the preparation pays off.
- Rollout, immutable style: spin up new machines on the new code, either in the existing cluster (needs session stickiness) or in a new cluster checked for health before switching the IP address (the IP switch may be traumatic to unfinished requests). With very frequent deployments, prefer the existing cluster: no interrupted connections.
- Under every model in-memory session data is lost and users must not notice: it “should only be a local cache of information available elsewhere”. “Decouple the process lifetime from the session lifetime.” Wait and watch monitoring first.
- Cleanup: “a job isn’t done until the tools are put away”. Delete shims in a new migration. Then contraction: drop old tables, views, columns, aliases or synonyms and uncalled stored procedures; apply NOT NULL and foreign key constraints. Constraints come only after rollout, because the old version cannot satisfy them and would start throwing errors, which “breaks our principle of undetectability”.
- A migrations framework keeps each change as a version-controlled asset and applies those missing from the schema. The older style (a modeling tool or DBA producing one SQL file) needs more effort: model the expansions, version, then model the contractions. The time-ordered sequence of changes is a common way to test them everywhere.
- Schemaless cleanup deletes unused documents, keys or elements. Also review feature toggles (new ones default to “off”) and schedule unneeded ones for removal.

Deploy Like the Pros

- In the late 2000s, developers delivered a binary and a readme and operations went to work. No longer: deployments are frequent and should be seamless, and the boundary between operations and development has become “fractal”. “We must design our software to be deployable, just as we design software for production.”
- Designing for deployment “gives you the ability to make large changes in small steps”, resting on automated action and quality checking.
- Example: forgetting an index on a foreign key constraint causes hours of downtime. Blaming the DBA for not checking is the dark side. The better answer starts from the premise that the pipeline catches all mechanical errors, so specify schema changes as data (a home-grown DSL or off-the-shelf migration library) rather than SQL DDL, giving the pipeline “X-ray vision” to reject any build with a foreign key and no index.
- “Have the humans define the rules. Have the machines enforce them.” It lets the team sleep instead of praying to the Polycom (back to the Godot chapter).

Wrapping Up

- To be successful, software will be deployed early and often, so deployment is an essential part of the system’s life and worth designing for. “Zero downtime is the objective.”
- Smaller, easier deployments allow big changes in a series of small steps, reducing disruption to users, human or program.
- This was the “interior” view; next is how the software fits the ecosystem, starting with protocol versions.

> **How this chapter builds the author’s case.** It answers the Godot chapter’s deployment army by making deployment part of the design, and draws on Chapter 8 (configuration management and immutable infrastructure) and on the book’s premise that 1.0 is only the beginning. It hands off to protocol versions with other systems.

CHAPTER 14

Handling Versions

> **Place in the argument — why this chapter, why this title.** The title is about versions at the boundary between your software and others'. It follows chapter 13 because frequent, easy deployment also means easily and repeatedly changing "the way our software talks with the rest of the world". The sub-headings run outward-in: first your obligations as a provider (Help Others Handle Your Versions), then your defense as a consumer (Handle Others' Versions). The goal is that the software be "a good citizen".

Chapter opening — untitled section before the first sub-heading in the book

- Because we can deploy often, we can change how our software talks to the world often. Each change risks breaking consuming applications, which forces other teams to do more work.
- Better for everyone if we do "some extra work on our end to maintain compatibility rather than pushing migration costs out onto other teams".

Help Others Handle Your Versions

- Consumers have their own teams and schedules. "If you want others to respect your autonomy, then you must respect theirs." You cannot make them release when you do, whether SaaS, inside one organization or across a partner channel. Coordinating provider and consumer deployments "doesn't scale"; follow the ripple effect and the whole company must upgrade at once. So most new versions of a service should be compatible.
- Nonbreaking API Changes: Postel's robustness principle, "Be conservative in what you do, be liberal in what you accept from others." It has mostly worked for the Internet, with caveats from Chapter 11.
- An API is "a layered stack of agreements". We can assume IP, TCP, UDP and DNS; above that, at layer 7, parties must agree on connection handshaking and duration, request framing, content encoding, message syntax, message semantics, and authorization and authentication. HTTP bakes in some, but its specification is vast; services "agree to a subset of the standard".
- So a breaking change is "any unilateral break from a prior agreement": rejecting a protocol, framing, encoding, syntax or routing that worked before; adding required request fields; forbidding optional request information previously allowed; removing guaranteed response information; requiring more authorization.
- The robustness principle makes requests and responses asymmetric (covariant requests, contravariant responses, Liskov). We can always accept more or return more than before, never accept less, require more, or return less.
- Always safe (Rich Hickey's set view): require a subset of the previously required parameters; accept a superset of those previously accepted; return a superset of the values previously returned; enforce a subset of the previously required constraints. Machine-readable specs let you verify this by comparing new with old.
- The tough problem is the gap between what the service says it accepts and what it accepts. Example: a url field never validated as a URL. Adding validation is a breaking change, though the documentation said to pass a URL. "As soon as the service went live, its implementation becomes the de facto specification." Tightening is not allowed: the principle leaves "no choice but to keep accepting the input".
- Likewise behavior that just happens to work, such as an edge case or an empty collection instead of an omitted element: once the service is public "a new version cannot reject requests that would've been accepted before".
- Still publish message formats (Swagger/OpenAPI), so consumers can code to the spec and generated tests can probe it (inbound testing, finding spec-versus-belief and spec-versus-implementation

gaps). Run randomized outbound tests against services you consume, to check your understanding of their spec.

- His example: as consumer, his team wrote FIT tests of every case in a shared spec and ran them against the other team's staging system. Almost 100 percent failed at first, which made the spec specific; afterward, no operational failures in a year. These "contract tests" exercise the parts of the provider's contract the consumer cares about, and being owned by the caller they are an early warning system.
- Breaking API Changes: when nothing else will suffice, first put a version number in request and reply formats, the version of the format, not the application. It is for debugging, not automatic bridging, since a consumer supports one version at a time.
- Running example: a peer-to-peer lending service with application and borrower routes (Table 1). New requirements (richer UI, languages, currencies, one entity as both borrower and lender) break the returned data and the borrower routes. "HTTP gives us several options to deal with these changes. None are beautiful."
- Options. (1) Version in the URL: most common, easy to route, share and see in logs; but one entity looks like different resources, "a big no-no in the REST world". (2) Versioned media types in Accept and Content-Type: stored URLs keep working, but the URL alone is not enough and clients need to know the media types. (3) A custom header such as "api-version": flexible, but needs routing helpers and is more secret knowledge. (4) A version field in PUT and POST bodies: easy, but does not cover all cases.
- He usually puts the version in the URL: the URL alone is enough, and caches, proxies and load balancers need no special, "error-prone" configuration. Useful when the next revision changes framework and he wants a separate cluster.
- Whatever the choice, support old and new side by side for a period (zero-downtime deployment, of course) so consumers upgrade as they are able. Test mixed old and new calls on the same entities; new-created ones often fail via the old API. If the version is in URLs, bump all routes at once.
- Internally, he handles it in the controller. New-API methods go straight to the current business logic; old-API methods convert old objects to current ones on requests and new to old on responses.

Handle Others' Versions

- Receiving requests, "your application has no control over the format. None, zip, zero, nada, zilch." Some joker will send a bogus message. Calls out can be rejected at any time: "Always be defensive."
- About all you can expect is the right syntactic type per field, and only with an automatic mapping library; with raw JSON, not even that.
- Example: banks join the loan service, so you add creditScore, collateralCategory and a new riskAdjustments value. A caller may send all, some or none of them. Usually your function must accept any combination. What if the category says mobile home but risk adjustments are missing, or the credit score is missing? Putting fields in the spec does not mean you can assume anyone will obey.
- Calls to suppliers have the same problem: "A request that worked just a second ago may fail now."
- Contract testing helps again. A common failing is overspecified integration tests: set up a request, issue it and assert on the response from the original request's data. That verifies the loop works now, not that the caller conforms or can handle any allowed response, so an allowed but unexpected change breaks the consumer. Provoking error responses needs special flags that reach production.
- Preferred: each side checks its own conformance, split in two. The request side checks that requests are created according to the provider's requirements. The response side checks that the caller handles responses by generating fake responses. Neither invokes the external service; this isolates breakdowns and removes unjustified assumptions about the other party.

- “As always, your software should remain cynical.” Even a trusted provider needs protection (Chapter 5, Stability Patterns).

Wrapping Up

- Versioning is “inherently messy” and will remain complex. He recommends a utilitarian philosophy: net suffering is minimized if everyone “thinks globally and acts locally”. The alternative is an organization grinding to a halt as every release waits on synchronized client upgrades.
- We saw how to handle our versions to aid others and defend against others’ changes. Next: transparency.

How this chapter builds the author’s case. It extends chapter 13’s zero-downtime deployment from internal to external change (side-by-side versions use it) and reuses Postel’s principle with Chapter 11’s caveats. Its defensive stance points to Integration Points and Chapter 5’s stability patterns.

PART IV OVERVIEW

Part IV — Solve Systemic Problems

Author's description (Preface): the Part is about the system's ongoing life ("1.0 is birth"): evolutionary architecture, shared knowledge across systems, and antifragile systems through chaos engineering.

The theme of the Part

- A system must keep changing and surviving after release; resilience is built and tested, not assumed.

How the chapters build the theme

- Chapter 15, Case Study: Trampled by Your Own Customers shows earlier warnings failing together at launch.
- Chapter 16, Adaptation: "Change is guaranteed. Survival is not."; people, process, architecture and information.
- Chapter 17, Chaos Engineering: deliberate failure to learn how the system really behaves.

What the Part contributes to the book

- Closes the book by treating release as the start of the software's true life.

CHAPTER 15

Case Study: Trampled by Your Own Customers

> **Place in the argument — why this chapter, why this title.** The title names the twist: the site was not brought down by an attack or a marketing miscalculation but by its own customers and the visitors a real launch attracts. It is a case study placed in Part IV (Solve Systemic Problems), so the earlier patterns can be seen failing together in one story. The author says the site "wasn't" built with the stability patterns, and he refers back to Broken Authentication and Session Management (page 218). The sub-headings follow the clock: launch, the three years behind it, the load testing, the crash explained, the gap in testing, and the aftermath.

Chapter opening — untitled section before the first sub-heading in the book

- The author joined a team of more than three hundred, nine months before launch, to build a complete replacement for a retailer's online store, content management, customer service and order-processing systems.
- The system was meant to be the company's backbone for the next ten years and was already more than a year late. The team had been in crunch mode for months, some for more than a year.

Countdown and Launch

- The project had "at least six different 'official' launch dates", three months of load testing and emergency code changes, two management teams and three user-load targets, each revised downward.
- Marketing gathered for the launch ceremony; the technologists watched the health of the site on a wall of monitors. The program manager hit a big red button, but the real change was a scheduled CDN metadata update taking about eight minutes to spread worldwide. The conference-room browser bypassed the CDN and hit the origin servers directly.
- Traffic grew fast: 10,000 sessions by 9:05, more than 50,000 by 9:10, 250,000 by 9:30. "Then the site crashed."
- His verdict: the launch was a "big bang" release in the literal sense.

Aiming for Quality Assurance

- To explain the crash he looks back over three years. His premise: "there's no such thing as a website project". Every one is "really an enterprise integration project with an HTML interface".
- When back end and front end are built together, a tighter integration is possible but not automatic. It depends on Conway's law, and the more common result is that both sides "end up aiming at a moving target".
- Conway's law (sidebar): organizations that design systems produce designs that copy their communication structures; an interface can only be built if two people communicate about its specification. He uses it prescriptively (creating the communication structure he wants) and descriptively (mapping software to reveal the real one).
- Replacing the whole commerce stack at once carries technical risk. In a tightly coupled system without stability patterns, the chance of failure is the joint probability that any one component fails. This one lacked the patterns, and with a completely new stack "Capacity, stability, control, and adaptability are all giant question marks."
- The developers were building everything to pass testing, not to run in production. Across fifteen applications and more than five hundred integration points, every configuration file was written for the integration-testing environment. QA also assumed a topology unlike production (e.g. production had extra firewalls). Changing QA was costly, so most developers ignored the discrepancies.
- Nobody could say where production configurations were kept. He compiled a list of properties that might need to change (hostnames, ports, URLs, database parameters, log locations) and pressed developers for answers; even "host" was ambiguous.

- Thousands of files would be overwritten with every release, so hand-editing was “a nonstarter”. His fix was “a level of indirection”: overrides kept separate from the application codebase, with each environment-specific property in exactly one place. This also kept production database passwords out of QA and source control.
- Setting up production, he “had inadvertently volunteered to assist with the load test.”

Load Testing

- The client budgeted a full month for load testing, “longer than I’d ever seen”. Marketing’s launch requirement was 25,000 concurrent users.
- Counting concurrent users is “a misleading way of judging the capacity”. There is no long-standing connection, only discrete requests tied into a session. The server cannot tell a user who has left from one who has not yet clicked, so it applies a timeout. A session is therefore “absolutely guaranteed to last longer than the user”, and counting sessions overestimates users. Active sessions are one of the most important measurements, but are not a count of users.
- Rapid turnaround was needed, so a standing conference call held the test manager, a load-test vendor engineer, a development architect, a DBA and the author.
- “Load testing is both an art and a science.” Real traffic cannot be duplicated, so analysis, experience and intuition are used. The scripts mixed “grazers”, “searchers” and “buyers”: over 90 percent viewed the home page and one product page, and an optimistic 4 percent went through checkout, the most expensive action (external integrations, about twelve pages against seven). They believed the mix was slightly harsher than real traffic.
- First run: the site locked up at 1,200 concurrent users and every application server had to be restarted. Capacity had to improve twenty times.
- They were on the call twelve hours a day for three months. They found new bottlenecks daily, turned configuration changes around in a day, code in two or three days, and made a few major architecture changes in under a week.
- Result after more than sixty builds: tenfold capacity, 12,000 active sessions (about 10,000 customers). Over that the site no longer crashed but got “flaky”. Marketing lowered the target, preferring a slow site to no site, with major improvements expected before the holidays.

Murder by the Masses

- Demand was not the cause (customers had not been told the date), and the production and test environments were not the issue either. “It was the number of sessions that killed the site.” Sessions are “the Achilles’ heel of every application server”: each uses RAM, and with replication enabled each was serialized and sent to a backup server after every page request, costing RAM, CPU and bandwidth.
- The cause was noise. The load scripts mimicked real users with real browsers and cookies: “They were polite to the system.” The real world “can be rude, crude, and vile.”
- Search engines drove about 40 percent of visits, and on launch day sent customers to old-style URLs. The web servers sent all .html requests to the application servers, so each such visitor created a session just to get a 404. Engines then refetched cached pages. His aside: do not abandon your old URL structure.
- One search engine was creating up to ten sessions per second, from a security-team mandate to use query parameters instead of session cookies (see ‘Broken Authentication and Session Management’, page 218).
- Nearly a dozen high-volume scrapers and shopbots hid their origins (varying subnets and user-agent strings), would never respect robots.txt (the site had none), and handled no cookies, so each created sessions.
- “Random weird stuff”: one Navy-base computer kept re-requesting its last URL. They blocked it: “Better to lose that one customer than all the others.”

The Testing Gap

- Two things were missing. First, the tests used the application “the way it was meant to be used”. No script hit one URL without cookies 100 times per second; had one done so, “we probably would have called the test ‘unrealistic’ and ignored that the servers crashed.”
- Testers do not follow the happy path. Load testing should be the same: “Add noise, create chaos.” Noise might only bleed away some capacity, “but it might also bring your system down.”
- Second, the developers built no safety devices to cut off bad situations. Threads kept entering the danger zone and piled up behind hung ones, “Like a car crash on a foggy freeway”. They saw this from day one but blamed the test methodology rather than a deficiency in “the system’s ability to recover from insults.”

Aftermath

- The CDN engineers used edge scripting to add a gateway page: it redirected clients that mishandled cookies to an explanation page, applied a throttle on the percentage of new sessions allowed, and blocked specific IP addresses. An engineer watched session counts for three weeks; a saturated server took nearly an hour to recover. By week three the throttle stayed at 100 percent.
- The home page was fully dynamic and identical for everyone, served five million times a day, needing over 1,000 database transactions and traversal of eighty-odd categories, with personalization that marketing had not decided how to use. They built scripts to serve a static copy to unidentified customers.
- Lawyers sent cease-and-desist notices to identified bots; most stopped, but “They never stayed away for long”.
- Session failover, based on serialization, should hold only small data such as keys. The sessions held whole shopping carts and up to 500 search results, so “We had no choice but to turn off session failover.”
- Common themes: “nothing is as permanent as a temporary fix” (most lasted years, rolling restarts a decade); all cost tremendous money, chiefly lost revenue (throttled customers, checkouts sent back to the cart page, no personalization, doubled hardware with extra labor and licenses, and a year of remediation instead of new features).
- “The worst part is that no amount of those losses was necessary.” Two years later the site handled more than four times the load on fewer servers of the same model. Built that way originally, the engineers could have joined marketing’s party.

> **How this chapter builds the author’s case.** It assembles earlier warnings into one failure: a system without stability patterns, a session-ID decision already condemned on page 218, and threads with no safety devices to cut off bad situations. Its testing lesson is the author’s own: “Add noise, create chaos.”

CHAPTER 16

Adaptation

Place in the argument — why this chapter, why this title. The subtitle, “Change is guaranteed. Survival is not.”, names the stakes. Earlier chapters made a system survive the world (stability, capacity, security); this one asks whether it can change as the world changes. Nygard opens with the business reason, then follows his own path: “Our path touches people, processes, tools, and designs.” The sections run from the human side (Process and Organization), to the structure of the software (System Architecture), to the last source of inflexibility, how data is structured, passed and referred to (Information Architecture). Convex Returns comes first because it says when adaptation is worth pursuing at all. He says the four themes interrelate and must be introduced in parallel and incrementally.

Chapter opening — untitled section before the first sub-heading in the book

- The Silicon Valley mantras (“Software is eating the world.” and “Move fast and break things.”) share “a total focus on change”, either withstanding it or better, creating it.
- Agile embraced change in response to business conditions, but now the arrow often points the other way: software change creates new products and markets and new alliances and competition (light bulb makers running server-side software on a retailer’s cloud).
- Sometimes the competitor is “yesterday’s version of the product”, as in a startup racing for product-market fit before the cash runs out.
- In all these cases “we need adaptation”, the chapter’s theme.

Convex Returns

- Not every piece of software needs to mutate daily. Where each release needs expensive certification (avionics, implantable medical devices), there is an inescapable transaction cost per release.
- Exceptions exist (JPL hotfixed the Spirit rover), but they were stuck with the launch hardware: “No in-flight upgrades to the RAM!”
- The rule: “Rapid adaptation works when there’s a convex relationship between effort and return.” Competitive markets usually exhibit such convexities.

Process and Organization

- A company changes through a decision cycle: someone senses a need, someone decides a feature fits and how quickly it is worth doing, someone acts (builds and markets), someone sees whether the change had the expected effect, and it starts over. A small company may use one or two people; in a large one the roles of “observer,” “decider,” or “doer” are diffused, sometimes to whole committees.
- The time around this loop, from observation to action, “is the key constraint” on absorbing or creating change. Formalize it as Deming/Shewhart (plan-do-check-act), OODA (observe, orient, decide, act), or market experiments and A/B tests; faster is more competitive.
- This drives the fail fast motto for startups (better learn fast, or simply adapt) and corporate innovation labs. Reacting is not the goal: run the loop faster than competitors and you force them to react to you; you are then “inside their decision loop”.
- Agile and lean removed delay from “act”; DevOps removes more and adds tools for observing. The timer should start at the first observation, not when the story lands in the backlog. The next frontier is the deciding phase.
- The Danger of Thrashing: it “happens when your organization changes direction without taking the time to receive, process, and incorporate feedback” (shifting priorities, unending crises). Shortening cycle time below the speed of environmental feedback is the trap.
- His analogy is pilot-induced oscillation, “porpoising”: the pilot pulls back, the plane responds late, he keeps pulling, then overcorrects the other way. Thrashing is the same: it happens when feedback from the environment is slower than the rate of control changes.

- Remedy: a steady cadence of delivery and feedback. If one outruns the other, do not slow it; use the spare time to speed the other. If development outpaces feedback, build an experimentation platform rather than more deployment tools.
- Platform Team: infrastructure and operations became programmable (virtualization, cloud, containers), so, per the layers of Chapter 7, development and operations are needed "up and down the stack".
- The platform needs a team that views application development as its customer, offering API and command-line provisioning of the common capabilities (compute, workload management, storage, logs, metrics, queuing, traffic management, DNS, email, access control, per Chapter 10, Control Plane). Each could be built by individual teams but "they aren't valuable in isolation."
- The platform team implements mechanisms that let others do the real provisioning: it does not write your monitoring rules, it provides the API to install them; it builds the service that builds API gateways, not the gateways. A vendor Platform gives a massive head start but does not replace the team.
- It "must not be held accountable for application availability"; that stays with application teams. It is measured on the platform's own availability.
- Orientation must be customer-focused, a radical change from the old split where operations was the last line of defense and development "more of a suspect than a customer!" Rule of thumb: "if your developers only use the platform because it's mandatory, then the platform isn't good enough."
- The Fallacy of the "DevOps Team" (sidebar): a team between development and operations is an antipattern. It creates two interfaces where there was one, and DevOps is a cultural transformation (open sharing of information and skills, data-driven decisions), not deployment automation. Such a team is really a platform or tools team (say so) or an adoption team focused on education and evangelism.
- Painless Releases: the release process of Chapter 12 (Waiting for Godot) rivals NASA mission control, with over twenty people. Because releases are arduous there are few, so each is unique, needs more planning, and is more painful, which discourages more releases.
- Releases should be about as big an event as "getting a haircut". Beyond the business case, frequent releases force you to get "really good at doing releases and deployments." "A closed feedback loop is essential to improvement", and frequent incremental releases let you outpace competitors.
- Releases cost too much and carry too much risk. The easy, harmful answer is to slow the release calendar, like going to the dentist less frequently because it hurts. The right answer: reduce effort, remove people, automate and standardize.
- Patterns from Humble and Farley's Continuous Delivery: Canary Deploy (one instance under scrutiny, then the rest); Blue/Green Deploy (two pools; deploy to the idle one, test, shift traffic); deploying in waves for very large environments.
- All act as governors (see Governor) limiting the rate of dangerous actions, and limit how many customers can be exposed to a bug.
- Service Extinction: natural selection is wasteful, but its key ingredients are "repeated iteration of small variations with selection pressure"; the ecosystem persists though individuals do not. Evolutionary architecture (below) tries to capture this: independent variation in small grains makes the organization antifragile. "Paradoxically, the key to making evolutionary architecture work is failure": try different approaches and kill the less successful.
- Example: two promotion ideas, tracking-based targeting versus a blanket offer. One big promotions service accumulates complexity faster than two small ones (routing and precedence, overgeneralizing frameworkitis, a vicious cycle of more code and harder change, and a shared database with no isolation of failure domains). Instead build two services, user-based and page-based, each handling one dimension; the page one may be a table in code ("Just call your source

code repo the content management repository."). Essential complexity remains but is portioned out, like pruning "a bonsai juniper" rather than "a hundred-foot oak".

- The laggard has only five choices: keep both; defund the winner to fix the loser; retool the loser for a different area (a pivot); delete it; or shut down the company for a hot dog and doughnut shop in Fiji. Corporations pick #1 or #2 and starve the successes; shutting a service carries "a kind of moral stigma." #3 is better, but consider #4: "The most important part of evolution is extinction." Deleting frees capacity and reduces dependencies: "Kill services in small grains to preserve the larger entity."
- Team-Scale Autonomy: Bezos's two-pizza team is misunderstood. It is not just fewer people; a self-sufficient team must cover several disciplines, because it is "about reducing external dependencies", each one a Lilliputian rope on Gulliver.
- No Coordinated Deployments (sidebar): needing to update provider and caller of an interface at once is a warning of strong coupling; as provider, make changes backward-compatible (see Nonbreaking API Changes) or add a new route and keep the old one for days or weeks. Waiting on a DBA or on review boards adds queuing time; the aim is a small group that "can be self-sufficient and push things all the way through to production."
- That needs firewalls, load balancers and SANs wrapped in APIs; the platform team's objective "must be to enable and facilitate this team-scale autonomy."
- Beware Efficiency, first way: "fully utilized" people look efficient, but "Keep the people busy all the time and your overall pace slows to a crawl."
- Second way: the enlightened view looks at the work, aiming for short cycle time and high throughput, but "as you make a value stream more efficient, you also make it more specialized to today's tasks", harder to change later.
- Examples: a car maker's rig that holds the car from inside cut cycle time but needed a custom rig per model, so "Efficiency came at the cost of flexibility." A two-person sailboat can stop anywhere; a container ship docks only at deep water terminals. In software: porting a build pipeline. "All the hidden connections that make it efficient also make it harder to adapt."
- Applied to automation: shell scripts are the sailboat; a fully automated pipeline into Kubernetes is faster "at the cost of making some serious commitments". Use the company grapevine first (in 2017 many were uneasy about AWS dependency).
- Summary: "Adaptability doesn't happen by accident." The natural order of software is the Big Ball of Mud: dependencies proliferate and coupling draws systems into one brittle whole. Next, from the human side to the structure of the software.

System Architecture

- Petroski says "Form follows function" is false; the rule is "Form follows failure": even the paper clip changed by correcting flaws. A system has "gaps and protrusions" between its shape and the solution space it is meant to occupy.
- Evolutionary Architecture: Ford, Parsons and Kua define it as one that "supports incremental, guided change as a first principle across multiple dimensions." Yet many basic styles inhibit this.
- The typical layered enterprise application (user interface, session, domain, persistence) separates layers so technology can change on either side, but how often is the database swapped alone? "Layers enforce vertical isolation, but they encourage horizontal coupling": a few giant domain classes rule the world and every change ripples and forces retesting.
- Rotate the barriers 90 degrees and you get component-based architecture: components isolated from each other with narrow, formal interfaces, like microservices in one process. Each owns its whole stack, database to UI or API; the UI is federated by hyperlinks or a front-end app calling a gateway. A few such stacks form "self-contained systems." This allows guided change along "business requirements" and "interface technology."

- Other styles: microservices (small disposable units, team autonomy; coupled to platform for monitoring, tracing, delivery), microkernel and plugins (formal interfaces to extensions; vulnerable to language and runtime), event-based (asynchronous messages; vulnerable to semantic change in message formats).
- Every style has trade-offs; until an "Ur-architecture" evolves in every dimension, choose what matters. A hypergrowth startup values scaling the tech team over long-term evolution of requirements; an enterprise depreciating capital over five years must evolve along business requirements and the technology platform.
- A Note on Microservices (sidebar): "Microservices are a technological solution to an organizational problem." Communication paths and dependencies grow exponentially; class dependencies follow a power law, so changes hit hub classes ("action at a distance"), developers avoid them, and the code degrades to a Big Ball of Mud. Microservices curtail size to what fits in one developer's head, but services get orphaned when you downsize. "Don't pursue microservices just because the Silicon Valley unicorns are doing it."
- Loose Clustering: "the loss of an individual instance is no more significant than the fall of a single tree in a forest." So servers have no differentiated roles, or roles present in more than one instance; a unique role uses leader election. Members start in any order and depend on the other cluster only via a virtual IP or DNS name, never on individual instances.
- The "principle of ignorance" goes further: members are not configured with the identities of other members, which would hinder adding or removing members and encourage point-to-point communication, "a capacity killer." Members may discover colleagues at runtime (for leader election and failure detection) without static configuration. Result: each cluster scales independently.
- Explicit Context: given a bare item string in JSON, a service can only pass it on as a token, look it up via another service, look it up in its own database, or discard it. As a token, converting it from string to number is a mistake. To resolve it, the bare string does not say who is authoritative, so the service must already know whom to call: "That's an implicit dependency", limiting it to one provider.
- With {"itemID": "https://example.com/policies/029292934"} the URL still works as an opaque token and as something to resolve, but the service no longer bakes in a solitary authority. It also makes integration testing easier: supply test harness URLs instead of "test" versions of other services.
- The same holds inside services: Rails ActiveRecord's implicit database connection hinders using several databases. "Global state is the most insidious form of implicit context", configuration parameters included; it slows going from "one" to "more than one."
- Create Options: you cannot extend the finished Sydney Opera House, but you can add a wing to the Winchester Mystery House, where continuous change is the vision. Mechanically, a flat wall has the potential for a door and curved shells do not: "A flat wall creates an option."
- "Modular systems inherently have more options than monolithic ones" (a PC graphics card is a swappable module). Baldwin and Clark's Design Rules give six modular operators; every module boundary is an option to apply them.
- Splitting: break a module into submodules, with cross-dependencies minimized. Shipping example: split by function (shipments, cost, delivery estimates) in sequence, or by carrier in parallel, which gives failure isolation (one carrier down, the others still ship, given timeouts). "The key with splitting is that the interface to the original module is unchanged."
- Substituting: replace a module (UPS or FedEx for the home-grown calculator); only the interface portion the parent needs must match, and subtle bugs creep in.
- Augmenting and Excluding: adding or removing a module. A technical decomposition (database, HTML, API, glue) rarely lets you exclude much; modules for recommendations, ratings and price can each be excluded, so "The second decomposition offers more options."
- Inversion: lift functionality duplicated across modules to the top as a first-class concern, such as several services each doing A/B tests inconsistently replaced by one "experimentation" service

whose result services read from a request header. It “creates a new dimension for variation and can reveal a business opportunity…like the entire market for operating systems.”

- Porting: repurpose a module from another system. It adds a dependency and risks coupling (if its road map diverges you must substitute), and the caller should exercise its own calls through the production interface. Forking and deploying a new instance is another way.
- Baldwin and Clark: six operators can create any complex structure of modules, and the economic value of the system increases with the number of options. Use them as thinking tools: imagine three ways to split features, allow exclusion or augmentation, look for lurking inversions.
- Summary: loose clusters; an evolutionary architecture (microservices, messages, microkernels, “or something that doesn’t start with m”); asynchrony, as it helps combat the stability antipatterns; explicit context; create options for modular operations. The last source of inflexibility is how we structure, pass and refer to data.

Information Architecture

- It is “how we structure data”: data and metadata describing what matters. It is “not reality, or even a picture of reality” but “a set of related models” capturing some facets, so we choose which facets, what to leave out, and how concrete to be.
- Embedded in the relational/object paradigm, we see limits only with effort: relational databases answer “What is the value of attribute A on entity E right now?” but are weaker on history, awkward with graphs and hierarchies, and “downright terrible” at images, sound or video.
- “Who wrote Hamlet?”: a relational table represents facts and would not allow the Marlowe theory; an RDF triple store holds assertions, each with an implicit “Oh yeah, who says?” In event journals the event is primary and current state means the cumulative effect of everything that happened.
- Each paradigm “defines what you can and cannot express.” Your job: decide which facets matter, how to represent them, how the representation survives over time, and which concepts stay local versus shared. “Sharing concepts increases expressive power, but it also creates coupling that can hinder change.”
- Messages, Events, and Commands: Fowler notes the overloading of “event”: event notification (fire-and-forget, no response used), event-carried state transfer (replicating entities so others can work), event sourcing (all changes recorded as events), and CQRS (different structures for reading and writing, not the same as events but often on the command side).
- Event sourcing (helped by Kafka) makes events the authoritative record; slow to walk, so we keep views and snapshots. “The event journal is the only truth. The others are caches, optimized to answer a particular kind of question.”
- Versioning is hard with years of events: avoid closed formats like serialized objects, schema-driven code generation, a class per message type, or annotation-based mapping; prefer open formats like JSON or self-describing messages. “Treat the messages like data instead of objects”. Apply the Chapter 14 versioning principles: a message sender is “communicating with a future (possibly not-yet-written) interface”, while a reader “is receiving a call from the distant past”.
- Services Control Their Identifiers: a “catalog” service (one catalog is never enough, see Embrace Plurality) should not assign an owner to each catalog and take an owner ID on requests. Problems: it couples to one authority for users, so caller and provider share an authentication protocol that stops at the organization’s edge and raises the barrier to use; and one owner can have only one catalog.
- Instead remove ownership: any user creates a catalog, the service issues its identifier (a catalog URL will do), and the user supplies it afterward. The service acts “like a little standalone SaaS business.” Access control goes to a “policy proxy” mapping client ID to catalog ID, factoring ownership out of the service. “Services should issue their own identifiers. Let the caller keep track of ownership.”

- URL Dualism: like quoting a word to mention it, a URL is a reference to a representation of a value, like a pointer. It can be resolved, or passed along and stored as an identifier unresolved. Exploiting this "we can break a lot of dependencies that otherwise seem impossible."
- Retail example: after acquiring a brand, a front end routes by item ID prefix to two databases, but "two" is a ridiculous number; only zero, one and many make sense. With a URL as both identifier and resolvable resource, the front end can use services that did not exist when it was built (proxy, outbound API gateway to a partner, even static files), with an HTTP cache against latency.
- It is a variation of Explicit Context: URLs carry the context needed to fetch the representation, more flexible than plugging IDs into a URL template. Caution: do not request arbitrary URLs from external users (see Chapter 11, Security); encrypt URLs sent to users so you can verify what returns is yours.
- Embrace Plurality: the "Single System of Record" pattern says each concept originates in one authoritative system, but the hard part is agreeing on the concept. Nouns like customer, order, account have several definitions (contractual company, entitled to call support, owes or has paid money, a trade-show contact). "The truth is that a customer is all of these things": "customer" is one facet, how your organization relates to the entity, and sales, support and accounting each want different attributes, life cycles and creators. This was the bane of enterprise shared object libraries and is now of shared services.
- Also the "dark matter" issue: whatever does not fit the system of record's model goes into a covert database or nowhere. Instead think in terms of federated zones of authority with common interchange formats.
- Avoid Concept Leakage: an electronics retailer selling digital tracks needed to reprice hundreds of thousands of tracks from $0.99 to $0.89 overnight. Someone created a "price point" entity so changing one amount reprices all related tracks, an elegant match for users' conceptual model. But making it global would ripple through every downstream system for years with Rube Goldberg release coordination, though they only needed it because item data was now incomplete. It was a concept leaking out across the enterprise: upstream leverage, downstream incidental complexity; flattening price onto items when publishing would have served as well.
- Lesson: "There's no such thing as a natural data model, there are only choices we make"; exposing internal concepts "creates semantic and operational coupling that hinders future change."
- Summary: we model only aspects of reality; every paradigm makes some statements easy, some difficult, some impossible, so choose deliberately among relational, document, graph, key-value or temporal databases. Consider recording the change rather than the new state (we stored current state only because disk was scarce; "That's not our problem today!"). Identifier abuse causes coupling: have services issue identifiers and use URLs as token and address. Be careful exposing concepts.

Wrapping Up

- "Change is the defining characteristic of software." Adaptation begins with release: release is the beginning of the software's true life, "everything before that release is gestation."
- Either systems grow over time, adapting to their environment, or they decay until costs outweigh benefits and then die.
- Make change cost less and hurt less by planning releases to production as an integral part of the software, not designing for change inside the software while disregarding the act of making that change live in production.

How this chapter builds the author's case. It draws on earlier chapters: the layers of Chapter 7 (platform team), the Control Plane of Chapter 10, the release ordeal of Chapter 12's Waiting for Godot, the governor from the stability patterns (canary, blue/green and waves limit dangerous actions), the Security chapter (do not resolve arbitrary URLs), and Chapter 14 on versioning (nonbreaking changes, message versions). Asynchrony "helps combat the stability antipatterns." The final turn reframes the

book: release is the start of the software's true life, so planning production releases is part of the software itself.

CHAPTER 17

Chaos Engineering

> **Place in the argument — why this chapter, why this title.** The title names a discipline, and the opening imagines telling your boss you will kill boxes in production, to show how radical the idea sounds, "but not a crazy one". The sub-headings run from definition and why the whole system matters, to antecedents, the best-known example (Netflix), adopting your own, applying the idea to people, and the close.

Chapter opening — untitled section before the first sub-heading in the book

- Telling your boss you will kill some boxes in production might end in a visit from Human Resources. Killing instances is "a radical idea", and one technique in an emerging discipline called "chaos engineering".

Breaking Things to Make Them Better

- Chaos engineering is "the discipline of experimenting on a distributed system in order to build confidence in the system's capability to withstand turbulent conditions in production." It is empirical rather than formal: "We don't use models to understand what the system should do. We run experiments to learn what it does."
- It deals with distributed, often large-scale systems. Staging and QA are poor guides: Scaling Effects (page 71) showed that different ratios of instances change behavior qualitatively, and the same holds for traffic, since congested networks behave differently from uncongested ones. Staging is never a full-size replica of production.
- Many problems only show in the whole system (excessive retries, cascading failures, dogpiles, single points of failure). Testing components in isolation gives no confidence either: "like concurrency, safety is not a composable property."
- Example: a client has a 50 ms timeout; each of two providers averages 20 ms (99.9 percentile 30 ms), so either is safe alone. Called in sequence, a sizable percentage of calls break the window.
- Hence the whole-system view: chaos engineering "deals with emergent properties that can't be observed in the individual components."

Antecedents of Chaos Engineering

- It draws on cybernetics, complex adaptive systems, high-reliability organizations and especially resilience engineering.
- Dekker's "drift" (Drift into Failure): a system (people, technology, processes) sits among boundaries of economy, capacity and safety. Pressure for economic return and reluctance to work at the upper limit of productivity push it toward the safety boundary and the barriers against disaster.
- Airliner example: the altitude best for revenue leaves the least room for error. Likewise we push distributed systems to the limit, profitable "right up until the time a disruption occurs."

> *"Highly efficient systems handle disruption badly. They tend to break all at once."*

- Chaos engineering is the balancing force: optimize for availability and tolerance to disruption "in a hostile, turbulent world rather than aiming for throughput in an idealized environment."
- A second thread is measuring events that don't happen. Weinberg's "fundamental regulator paradox": a regulator's task is to eliminate variation, yet that variation is the source of information about its work, so the better it does, the less it learns about how to improve.
- The "Volkswagen microbus" paradox: you learn to fix what often breaks, not what rarely breaks. "We want a continuous low level of breakage to make sure our system can handle the big things."
- Taleb's Antifragile describes systems that improve from stress. Information systems don't naturally do so; we expect disorder and want enough of it in normal operation that we are not flummoxed. We

use chaos engineering “the way a weightlifter uses iron”: tolerable stress to increase strength over time.

The Simian Army

- Netflix’s “Chaos Monkey” wakes up every so often, picks an autoscaling cluster and kills an instance. The cluster should recover automatically; if not, the owning team must fix it.
- It was born in the migration to AWS and microservices. As services multiplied, availability was jeopardized by more components, so every cluster needed to autoscale and recover from the loss of any instance. How to keep every deployment robust “when hidden coupling is so easy to introduce”?
- Netflix chose both: stability patterns make instances likelier to survive, but no code keeps AWS from terminating an instance. Terminations happen often enough to hurt at scale but not often enough to test every deployment, so Netflix needed failures “to happen more often so that they became totally routine” (the agile adage: if something hurts, do it more often).
- Other monkeys followed: Latency, Janitor, Conformity and Chaos Kong, released as the “Simian Army”.
- Opt In or Opt Out? At Netflix chaos is opt-out: every production service is subject to it; waivers need sign-off, and exemption carries a stigma. Others use opt-in, with much lower adoption but possibly the only feasible route for a mature, entrenched architecture. He suggests a new adopter start with opt-in: less resistance, and success stories first.

Adopting Your Own Monkey

- When Chaos Monkey launched, developers were surprised by how many vulnerabilities it found, even in old services.
- Prerequisites. Your efforts “can’t kill your company or your customers”. Netflix had it easy (customers press play again); if every request is irreplaceably valuable, it is the wrong approach. “You must be able to break the system without breaking the bank!”
- Limit the “blast radius”: how many customers are affected and how badly. Pick victims by criteria. Trace a user and request through the tiers, and know whether the whole request succeeded; a success shows where redundancy saved it, a failure shows where it broke.
- Know what “healthy” looks like and from what perspective. Have a recovery plan.
- Designing the Experiment. Begin with a hypothesis, such as Chaos Monkey’s “Clustered services should be unaffected by instance failures”, which observation quickly invalidated. State it as invariants and externally observable behavior, with a healthy steady state. Check you can tell now whether the steady state holds, and decide what evidence rejects the hypothesis; a non-zero failure rate may not suffice.
- Injecting Chaos. Apply knowledge of the structure to inject: kill an instance (the “most basic and crude” kind), add latency, or make a call fail. Latency Monkey finds services that time out instead of using a fallback. Netflix’s failure injection testing (FIT) tags a request at the edge with a cookie saying a given call, G to H, will fail, and the call site reports failure without making it.
- Targeting Chaos. Random selection is “as good a process as any” at first, since most software has so many problems. Later it becomes a search problem in a sparse space: with n service-to-service calls there are 2 to the n possible fault sets, so a fault in a weekly partner data load causing a later 500 is unlikely to be found at random.
- Humans can target by thinking about how a successful request works: a tree of calls; kick out a support and “Either way we learn something.” Study the faults that did not become failures: “The system did something to keep that fault from becoming a failure.” We keep an edge over computers in abductive reasoning.
- Automate and Repeat. It’s not “fun and exciting”: in the best case it’s “totally boring”. On finding a weakness, fix that instance, then look for the same class elsewhere, and automate. “There’s such a thing as too much chaos”: don’t kill the last instance of a cluster, or fail G’s calls to H and every

fallback together. Platforms (Netflix's ChAP) control how much chaos, when, to whom and what is off-limits.

Disaster Simulations

- Chaos isn't only software faults: people get sick, quit, or are cut off by natural disasters. "What happens when your single point of failure goes home every evening?" High-reliability organizations use drills and simulations.
- A "business continuity" exercise can be run smaller: designate people as "incapacitated" and see if business continues. In a "zombie apocalypse simulation", 50 percent of people stay away for the day.
- As with Chaos Monkey, early runs reveal key processes that can't be done (a role only one person has). Record issues, review them like a post-mortem, and fix them.
- Don't combine fault injection with a zombie simulation on a first run. Have a code word to abort, in case "learning opportunity" becomes "existential crisis".

Wrapping Up

- "Chaos engineering starts with paradoxes. Stable systems become fragile." Dependencies creep in and failure modes proliferate "whenever you turn your back on the software."
- His closing point: "We need to break things" regularly and in a semicontrolled way, to make "the software and the people who build it more resilient."

How this chapter builds the author's case. It closes the book on the idea that resilience cannot be assumed: the earlier chapters' stability patterns make instances likelier to survive, and Netflix pairs them with deliberate failure. It builds on Scaling Effects (page 71) and turns the missing noise of the previous case study into method: hypotheses, small blast radius, automation. The final Wrapping Up ends the book on a paradox and a call to "break things", the software and the people alike.